

# **Hospital Inpatient - Meal Planning System**

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

## **BACHELOR OF TECHNOLOGY**

**In**

**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**&**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**

**By**

**Myadagoni Ramya Sree**

**2520030094**

**D Varun Teja Reddy**

**2520090221**

Under the Esteemed Guidance of

**Dr. D. Ramya Krishna**

**Assistant Professor**

**Department of Computer Science and Engineering**



## **Koneru Lakshmaiah Education Foundation**

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, [www.klh.edu.in](http://www.klh.edu.in)

**K L (Deemed to be) University  
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**&**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



### **Declaration**

The Project Report entitled **“Hospital Inpatient – Meal Planning System”** is a record of Bonafide work of **Myadagoni Ramya Sree - 2520030094, D Varun Teja Reddy – 2520090221** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

**Myadagoni Ramya Sree – 2520030094**

**D Varun Teja Reddy - 2520090221**

**K L (Deemed to be) University**

**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**&**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



## **CERTIFICATE**

This is certify that the mini project based report entitled “**Hospital Inpatient – Meal Planning System**” is a bonafide work done and submitted by **Myadagoni Ramya Sree - 2520030094, D Varun Teja Reddy – 2520090221** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025**.

**Signature of the Guide**

**Signature of the Course Coordinator**

**Signature of the HOD**

## ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr Yerragudipadu Subbarayudu**, and my guide **Dr D.Ramya Krishna**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)/Dr. Kaja Shareef (CSIT)**, Head of the **Department for Computer Science Engineering/ Department for Computer Science and Information Technology** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

**Myadagoni Ramya Sree – 2520030094**

**D Varun Teja Reddy - 2520090221**



## Abstract

The *Adaptive Nutrition Planning Platform for Smart Inpatient Healthcare* is an application designed to improve the efficiency and effectiveness of meal planning in hospital environments. Proper nutrition plays a key role in patient recovery, and it requires careful consideration of factors such as dietary restrictions, allergies, medical conditions, and required calorie intake. Traditional methods of nutrition planning are often time-consuming, prone to errors, and lack proper coordination between healthcare staff and kitchen teams.

This platform provides an advanced and organized approach to hospital nutrition planning by considering all essential factors. It includes features such as patient dietary profile creation, intelligent meal planning, and automated kitchen order processing. The system helps in creating personalized nutrition plans for patients based on their medical conditions while ensuring that all dietary restrictions are properly followed. In addition, it supports storing historical dietary data, exporting information for analysis, and managing patient nutrition records efficiently.

The application also includes an administrative portal that allows easy monitoring of patient data and smooth handling of daily operations related to nutrition planning. It is built using modern technologies such as HTML5, CSS3, JavaScript (ES6+), React.js, Node.js, and RESTful APIs, ensuring a responsive and user-friendly experience.

Overall, the platform helps reduce human errors in meal planning and improves coordination between healthcare professionals and kitchen staff. By automating important processes and supporting better decision-making, it contributes to improved patient care and more efficient hospital operations.

## Index

| S. No. | Chapters                 | Topics                                                                                                                                                                                                                                                         | Page.no |
|--------|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
|        |                          | Acknowledgement                                                                                                                                                                                                                                                |         |
|        |                          | Abstract                                                                                                                                                                                                                                                       |         |
| 1      | Introduction             | 1.1 Background of the project<br>1.2 Problem statement<br>1.3 Scope of the project<br>1.4 Objective(s)<br>1.5 Importance of the application<br>1.6 Target users/audience                                                                                       | 9 - 19  |
| 2      | System Requirements      | 2.1 Hardware requirements<br>2.2 Software requirements<br>2.3 Development tools and frameworks                                                                                                                                                                 | 20 - 24 |
| 3      | Technology Stack         | 3.1 Front-end (e.g., React.js, Angular, HTML/CSS)<br>3.2 Back-end (e.g., Node.js, Django, Spring Boot)<br>3.3 Database (e.g., MongoDB, MySQL, PostgreSQL)<br>3.4 Version Control (e.g., Git, GitHub)<br>3.5 APIs or External services (e.g., Firebase, Stripe) | 25 - 30 |
| 4      | System Architecture      | 4.1 High-level architecture diagram<br>4.2 Description of each layer (frontend, backend, database)<br>4.3 Deployment architecture (e.g., cloud, local, CI/CD pipelines)                                                                                        | 31 - 33 |
| 5      | Design                   | 5.2 ER Diagram / Database schema                                                                                                                                                                                                                               | 34      |
| 6      | Implementation           | 6.1 Module-wise implementation<br>6.2 Front-end logic (UI rendering, state management)<br>6.3 API integration<br>6.4 Authentication & Authorization                                                                                                            | 35 - 41 |
| 7      | Features                 | 7.1 List of main features<br>7.2 How each feature works (user perspective + technical description)                                                                                                                                                             | 42 - 45 |
| 8      | Testing                  | 8.1 Postman testing (frontend/backend)<br>8.2 Integration testing<br>8.3 Tools used for testing<br>8.4 Test cases and results                                                                                                                                  | 46 - 50 |
| 9      | Deployment               | 9.1 Steps to deploy<br>9.2 Environment configuration<br>9.3 Hosting of frontend and backend                                                                                                                                                                    | 51 - 55 |
| 10     | Challenges & Limitations | 10.1 Issues faced during development<br>10.2 Solutions applied<br>10.3 Current limitations or known bugs                                                                                                                                                       | 56 - 59 |
| 11     | Future Enhancements      | 11.1 Planned features<br>11.2 Possible integrations or optimizations                                                                                                                                                                                           | 59 - 62 |
| 12     | Conclusion               | 12.1 Summary of the project<br>12.2 What was achieved<br>12.3 Skills learned during development                                                                                                                                                                | 63 - 67 |
| 13     | References               | - Books, tutorials, APIs, documentation sites used                                                                                                                                                                                                             | 68 - 69 |

|    |            |                                                                                                                                                                                                                                                                 |         |
|----|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| 14 | Appendices | <ul style="list-style-type: none"><li>- Screenshots of the app</li><li>- Sample code snippets</li><li>- Installation/setup instructions</li><li>- User manual or guide</li><li>-Geo Tag photos with guide</li><li>-Review forms with guide signatures</li></ul> | 70 - 78 |
|----|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|

## CHAPTER -1 INTRODUCTION

### 1.1 Background of the Project

Healthcare institutions place significant importance on patient nutrition, as proper dietary management directly influences recovery, treatment effectiveness, and overall patient well-being. In many hospitals, meal planning for inpatients is still handled through manual methods such as paper records, verbal communication, and handwritten diet charts. These traditional approaches often create challenges, including delays in meal preparation, inaccurate dietary assignments, poor coordination between departments, and increased chances of human error. Managing different diet types, calorie requirements, allergies, and food restrictions for numerous patients simultaneously becomes a complex and time-consuming task. As hospitals continue to expand their services, the need for a more organized and automated solution has become increasingly evident.

Advancements in web technologies have enabled the development of digital healthcare management systems that improve efficiency, accuracy, and communication within hospitals. The Hospital Inpatient Meal Planning System (HIMPS) was developed to address the limitations of conventional meal management processes by providing a centralized platform for dietary planning and nutrition management. The system allows healthcare professionals to maintain patient dietary profiles, calculate nutritional requirements, create personalized meal plans, perform allergy checks, monitor kitchen orders, and generate reports. By integrating these functionalities into a single platform, HIMPS helps reduce manual effort, improve patient safety, and ensure that dietary recommendations are followed accurately. The project demonstrates how modern web technologies can be applied to healthcare environments to enhance operational efficiency and support better patient care.

This project is built using HTML5, CSS3, JavaScript, Node.js, and Express.js, providing a user-friendly and responsive interface for different hospital staff members, including administrators, dietitians, nurses, and kitchen personnel. Through role-based access control and automated workflows, the system streamlines communication between departments and minimizes the risks associated with manual meal planning. As a result, HIMPS contributes to the digital transformation of hospital nutrition management while promoting safer, faster, and more reliable dietary services for patients.

## 1.2 Problem Statement

In modern healthcare environments, patient nutrition is a critical component of treatment and recovery. Proper meal planning ensures that patients receive the necessary nutrients required to support their medical conditions, enhance recovery rates, and maintain overall health. However, many hospitals and healthcare institutions continue to rely on traditional methods of managing inpatient dietary requirements, including paper-based records, manual calculations, spreadsheets, and verbal communication between departments. These methods often result in inefficiencies, communication gaps, and an increased risk of errors that can negatively affect patient care.

One of the major challenges faced by hospitals is the management of diverse dietary requirements for patients. Every patient has unique nutritional needs based on factors such as age, medical condition, allergies, dietary restrictions, treatment plans, and recovery goals. Dietitians are required to carefully assess each patient's condition and prepare suitable meal plans. In a manual system, maintaining and updating this information for a large number of patients becomes difficult and time-consuming. As patient records increase, healthcare staff may struggle to track all dietary requirements accurately, leading to inconsistencies in meal planning and service delivery.

Another significant issue is the risk associated with food allergies and dietary restrictions. Patients may have allergies to specific food ingredients such as gluten, dairy products, nuts, shellfish, soy, or eggs. If these allergies are not properly recorded, communicated, or verified during meal preparation, patients may receive meals that contain harmful ingredients. Such mistakes can result in severe allergic reactions, medical complications, or even life-threatening situations. In many hospitals, allergy verification is still dependent on manual checking, which increases the possibility of human error. Therefore, there is a need for an automated mechanism that can detect conflicts between patient allergies and meal ingredients before food is served.

Calorie and nutritional management present another challenge within hospital meal planning. Different patients require different calorie levels and nutritional compositions depending on their medical conditions and treatment objectives. For example, diabetic patients require controlled carbohydrate intake, while post-surgery patients often require high-protein diets to support healing. Dietitians frequently perform calorie calculations manually, which can be both time-consuming and prone to calculation errors. Incorrect calorie estimations may result in patients receiving insufficient or excessive nutritional intake, potentially affecting their recovery process.

A system capable of automatically calculating calorie requirements and nutritional targets would significantly improve the accuracy and efficiency of dietary planning.

Communication among healthcare departments is also a common concern. Effective meal management requires coordination between dietitians, nurses, administrators, and kitchen staff. In traditional environments, dietary instructions are often communicated through paper forms, phone calls, or verbal messages. This approach can lead to misunderstandings, misplaced records, delayed meal preparation, and incorrect meal delivery. Kitchen staff may not receive updated dietary instructions on time, resulting in patients receiving meals that do not align with their prescribed nutritional plans. Consequently, hospitals require a centralized platform that enables seamless communication and information sharing among all stakeholders involved in meal management.

Meal plan approval and monitoring processes are often unstructured in manual systems. Dietitians may create meal plans that require review and approval by senior medical staff or administrators before implementation. Without a formal workflow, it becomes difficult to track the status of meal plans, identify pending approvals, or maintain accountability. Delays in approvals can affect meal preparation schedules and patient care. Furthermore, hospitals may find it challenging to maintain records of who created, approved, modified, or rejected a meal plan. A structured approval mechanism is therefore essential to improve transparency and accountability.

The management of kitchen operations is another area that requires improvement. Once meal plans are finalized, kitchen staff must prepare and deliver meals according to patient-specific requirements. In traditional systems, meal orders are often maintained on paper or communicated verbally. This makes it difficult to monitor preparation progress, delivery status, and order completion. Lack of visibility into kitchen operations may result in delays, missing meals, duplicate orders, or delivery errors. An automated kitchen order tracking system would help ensure timely meal preparation and improve operational efficiency.

Hospitals also face challenges related to data management and reporting. Maintaining patient dietary records, meal histories, allergy information, and kitchen orders manually requires significant effort. Retrieving historical data for audits, quality assurance, or administrative purposes can be time-consuming and inefficient. Additionally, the absence of centralized reporting mechanisms limits the ability of hospital management to analyze nutrition services and

identify areas for improvement. Digital record management and report generation capabilities are therefore necessary to support informed decision-making and regulatory compliance.

Another limitation of existing systems is the lack of audit trails and activity monitoring. In healthcare environments, it is important to maintain records of all actions performed by users, including updates to dietary profiles, meal plans, approvals, and patient information. Manual systems provide limited visibility into these activities, making it difficult to investigate errors or maintain accountability. An audit logging mechanism can help track user activities and provide a reliable record of system operations.

Furthermore, many hospitals aim to adopt digital transformation initiatives to improve efficiency and service quality. However, the absence of specialized nutrition management systems often forces healthcare providers to depend on fragmented tools that are not designed specifically for hospital dietary operations. This results in duplicate data entry, increased workload, and reduced productivity. A comprehensive, integrated solution is needed to streamline dietary management processes and support modern healthcare requirements.

To address these challenges, the Hospital Inpatient Meal Planning System (HIMPS) has been developed as a centralized web-based application that automates dietary profile management, meal planning, allergy verification, calorie calculation, kitchen order tracking, approval workflows, notifications, and report generation. By replacing manual processes with a digital platform, the system aims to improve patient safety, reduce operational errors, enhance communication among departments, and ensure efficient delivery of nutritional services. The implementation of HIMPS contributes to better healthcare outcomes by providing accurate, organized, and technology-driven inpatient meal management.

### **1.3 Scope of the Project**

The scope of the Hospital Inpatient Meal Planning System (HIMPS) focuses on providing a comprehensive digital platform for managing hospital dietary and nutrition-related activities. The system is designed to support healthcare professionals in planning, monitoring, and delivering personalized meals to inpatients based on their medical conditions, nutritional requirements, allergies, and dietary restrictions. By integrating multiple nutrition management functions into a single application, the project aims to improve efficiency, accuracy, and patient safety within hospital environments.

The primary scope of the system includes patient dietary profile management. Healthcare staff can maintain detailed records of patient information, including diet type, calorie requirements, allergies, food restrictions, and nutritional goals. These profiles serve as the foundation for generating customized meal plans that align with each patient's medical needs. The system supports various diet categories such as diabetic, low-sodium, high-protein, renal, cardiac, vegetarian, and general diets, allowing flexibility in nutritional planning.

Another important aspect of the project is meal planning and scheduling. The system enables dietitians to create daily meal plans consisting of breakfast, lunch, dinner, and snack options. Meals can be selected from a centralized meal library that contains nutritional information, calorie values, diet compatibility, and allergen details. The meal planning module ensures that dietary recommendations are followed consistently and helps maintain proper nutritional balance for patients throughout their hospital stay.

The project also includes an automated calorie calculation feature that assists healthcare professionals in determining patient nutritional requirements. By considering factors such as age, weight, height, gender, activity level, and medical condition, the system calculates daily caloric needs and nutritional targets. This functionality reduces manual calculations and improves the accuracy of dietary planning.

A significant scope of HIMPS is allergy and dietary restriction management. The system provides an allergy checking mechanism that automatically compares patient allergies with meal ingredients before meals are assigned. This helps prevent dietary conflicts and minimizes the risk of allergic reactions. The feature contributes directly to patient safety and ensures compliance with prescribed dietary restrictions.

Kitchen order management is another major component within the scope of the project. Approved meal plans are converted into kitchen orders that can be tracked throughout different stages, such as pending, preparing, and delivered. Kitchen staff can monitor meal preparation activities, while healthcare personnel can verify delivery status, ensuring efficient coordination between departments.

The system further supports approval workflows, notifications, audit history tracking, and report generation. Administrators can review meal plans, manage user accounts, monitor system activities, and generate reports related to patients, meal plans, allergies, and kitchen operations. These capabilities enhance transparency, accountability, and decision-making within the hospital.

The current scope is limited to inpatient meal management and nutrition services within a hospital environment. Features such as online patient portals, mobile applications, electronic medical record integration, advanced analytics, and cloud-based deployment are not included in the present version. However, these functionalities can be incorporated in future enhancements. Overall, HIMPS provides a centralized, efficient, and user-friendly solution for hospital dietary management while laying the foundation for future expansion and integration with broader healthcare systems.

## **1.4 Objectives of the Project**

The primary objective of the Hospital Inpatient Meal Planning System (HIMPS) is to develop a centralized and efficient digital platform that automates the management of inpatient nutrition and dietary services within hospitals. The system is designed to reduce the dependency on traditional paper-based methods and provide a streamlined approach for planning, monitoring, and delivering patient meals. By integrating patient dietary information, meal planning, calorie calculation, allergy verification, and kitchen order management into a single application, the project aims to improve healthcare service quality and enhance patient safety.

One of the key objectives of the project is to maintain comprehensive dietary profiles for hospitalized patients. The system allows healthcare professionals to store and manage essential nutritional information such as diet type, calorie requirements, allergies, dietary restrictions, and medical conditions. This information helps dietitians and healthcare staff make informed

decisions while planning meals and ensures that every patient receives nutrition that aligns with their treatment plan and health requirements.

Another important objective is to automate the meal planning process. In traditional systems, preparing individualized meal plans for multiple patients can be time-consuming and prone to errors. HIMPS aims to simplify this process by providing a structured meal planning module that enables dietitians to create and manage daily meal schedules efficiently. The system supports the selection of meals from a predefined meal library containing nutritional information, making it easier to design balanced and medically appropriate diets.

Ensuring patient safety is also a major objective of the project. Food allergies and dietary restrictions are critical factors that must be considered during meal preparation. The system incorporates an allergy-checking mechanism that automatically compares patient allergies with meal ingredients before assigning meals. This helps prevent accidental exposure to allergens and reduces the risk of adverse health reactions, thereby improving the overall quality of patient care.

The project also aims to improve nutritional assessment through automated calorie calculations. By using patient-specific data such as age, gender, weight, height, activity level, and medical condition, the system calculates daily calorie requirements and nutritional targets. This feature reduces manual calculations, improves accuracy, and supports the creation of personalized nutrition plans that contribute to patient recovery and well-being.

Another objective is to establish effective communication and coordination between hospital departments. HIMPS connects dietitians, nurses, administrators, and kitchen staff through a shared platform where dietary information, meal plans, approvals, and kitchen orders can be accessed and updated in real time. This reduces communication delays, minimizes misunderstandings, and ensures that all departments work with consistent and up-to-date information.

The system further aims to provide a structured approval workflow for meal plans. Administrators and authorized personnel can review, approve, or reject meal plans before implementation. This process improves accountability, maintains quality standards, and ensures that dietary recommendations comply with hospital policies and patient requirements.

In addition, the project seeks to improve operational efficiency through kitchen order tracking and monitoring. The system enables kitchen staff to receive organized meal orders and update

their preparation and delivery status. This helps streamline food service operations and ensures timely meal delivery to patients.

Finally, HIMPS aims to provide reporting, audit tracking, and administrative management features that support data analysis, performance monitoring, and informed decision-making. Through these objectives, the project contributes to the digital transformation of hospital nutrition management and promotes a safer, more accurate, and efficient healthcare environment.

## **1.5 Importance of the Application**

In modern healthcare systems, proper nutrition management is essential for patient recovery, treatment effectiveness, and overall well-being. Hospitals are responsible for providing meals that meet the specific dietary requirements of patients based on their medical conditions, nutritional needs, allergies, and treatment plans. Managing these requirements manually can be difficult, especially in hospitals with a large number of patients. The Hospital Inpatient Meal Planning System (HIMPS) plays a significant role in addressing these challenges by providing a centralized and automated solution for hospital nutrition management.

One of the major benefits of the application is its ability to improve patient safety. Patients often have food allergies, dietary restrictions, and special nutritional requirements that must be carefully considered when preparing meals. In manual systems, there is always a possibility of human error, which can result in patients receiving inappropriate meals. Such mistakes may lead to allergic reactions, health complications, or delays in recovery. HIMPS minimizes these risks through automated allergy checking and dietary profile management, ensuring that meals assigned to patients are safe and suitable for their medical conditions.

The application is also important because it enhances the efficiency of hospital operations. Traditional paper-based meal planning requires significant effort from dietitians, nurses, and kitchen staff. Information must be manually recorded, updated, and communicated across departments, which can lead to delays and inconsistencies. HIMPS automates these processes and allows users to access patient dietary information, meal plans, and kitchen orders through a single platform. This reduces administrative workload and allows healthcare professionals to focus more on patient care rather than paperwork.

Another important aspect of the application is its support for personalized nutrition management. Every patient has unique dietary needs depending on factors such as age, gender, medical condition, activity level, and treatment requirements. The system enables dietitians to create customized dietary profiles and meal plans that match individual nutritional goals. The integrated calorie calculator further assists in determining accurate calorie requirements, helping healthcare providers deliver balanced and medically appropriate nutrition.

HIMPS also improves communication and coordination among different hospital departments. Effective nutrition management requires collaboration between dietitians, nurses, administrators, and kitchen staff. The system ensures that all stakeholders have access to accurate and up-to-date information regarding patient diets and meal schedules. This eliminates communication gaps, reduces misunderstandings, and promotes smooth workflow management throughout the hospital.

The application contributes to better monitoring and accountability through approval workflows, notifications, and audit tracking. Meal plans can be reviewed and approved before implementation, ensuring compliance with hospital standards and dietary guidelines. The audit history feature maintains records of system activities, making it easier to monitor changes, investigate errors, and maintain transparency in operations.

Another significant advantage is the availability of reporting and data management capabilities. Hospital administrators can generate reports related to patient diets, meal plans, allergies, kitchen orders, and system activities. These reports support decision-making, performance evaluation, and future planning. Digital record management also improves data accessibility and reduces the risk of losing important information.

Overall, the Hospital Inpatient Meal Planning System is an important healthcare application that improves patient safety, enhances nutritional care, increases operational efficiency, strengthens communication between departments, and supports effective hospital management. By replacing manual processes with an automated digital solution, HIMPS contributes to higher-quality healthcare services and better patient outcomes while supporting the modernization of hospital nutrition management systems.

## 1.6 Target Users / Audience

The target users of the **Hospital Inpatient Meal Planning System (HIMPS)** span a wide range of healthcare professionals involved in patient nutrition management and hospital food service operations. The system is designed to provide a centralized platform that improves communication, enhances efficiency, and ensures accurate dietary planning for hospitalized patients. By integrating multiple functions into a single application, HIMPS serves different stakeholders who contribute to patient care and nutrition management.

The primary users of the system are **Dietitians and Nutrition Specialists** who are responsible for assessing patient nutritional requirements and preparing appropriate meal plans. The platform enables them to create dietary profiles, calculate caloric requirements, manage food allergies, define dietary restrictions, and design personalized meal schedules. With access to nutritional data and meal information, dietitians can ensure that patients receive meals that align with their medical conditions and recovery goals.

The secondary users include **Nurses and Healthcare Staff** who monitor patients on a daily basis. Nurses can access patient dietary information, verify allergies, review meal plans, and ensure that prescribed meals are delivered correctly. The system helps them quickly retrieve nutrition-related information without relying on paper records, thereby reducing errors and improving patient safety.

Another important group of users is **Kitchen Staff and Food Service Personnel**. These users are responsible for preparing and delivering meals according to approved dietary plans. HIMPS provides them with organized kitchen orders containing detailed information about meal requirements, patient locations, and delivery schedules. This ensures smooth coordination between the nutrition department and food preparation teams while improving operational efficiency.

**Hospital Administrators and Management Personnel** also benefit from the system. They can manage user accounts, monitor system activities, review reports, oversee meal plan approvals, and track kitchen operations. The centralized dashboard allows administrators to make informed decisions and maintain accountability across all nutrition-related processes within the hospital.

Additionally, healthcare organizations and hospitals seeking to modernize their nutrition management services can use HIMPS as a digital solution for improving patient care and operational effectiveness. Educational institutions and researchers working in healthcare technology, nutrition management, and hospital information systems can also use the project as a reference model for academic study and research purposes.

In essence, the platform caters to:

1. **Dietitians and Nutrition Specialists** seeking efficient dietary planning and nutrition management.
2. **Nurses and Healthcare Staff** requiring quick access to patient dietary information and allergy details.
3. **Kitchen Staff and Food Service Teams** responsible for meal preparation and delivery tracking.
4. **Hospital Administrators** managing users, approvals, reports, and overall system operations.
5. **Healthcare Organizations and Researchers** interested in digital healthcare solutions and nutrition management systems.

Overall, the target audience represents a collaborative network of healthcare professionals who work together to ensure effective patient nutrition and meal management. The system bridges communication gaps between departments, improves patient safety, reduces manual workload, and supports the delivery of high-quality healthcare services through an integrated and user-friendly platform.

## CHAPTER -2 SYSTEM REQUIREMENTS

### 2.1 Hardware Requirements

The Hospital Inpatient Meal Planning System (HIMPS) is a web-based application designed to operate efficiently on standard computer systems commonly available in hospitals, clinics, and healthcare organizations. Since the application is developed using modern web technologies such as HTML, CSS, JavaScript, Node.js, and Express.js, it does not require specialized hardware for deployment or operation. The system is designed to be lightweight, user-friendly, and accessible through standard web browsers, making it suitable for both small and large healthcare environments.

A reliable processor is essential for ensuring smooth system performance. The application can operate effectively on systems equipped with an Intel Core i3 processor or equivalent. However, for improved multitasking and faster performance, an Intel Core i5 or higher processor is recommended. The processor is responsible for handling application execution, data processing, and user interactions while ensuring quick response times.

Memory (RAM) is another important hardware component. A minimum of 4 GB RAM is required for running the system efficiently. This allows users to access multiple browser tabs, manage patient records, and perform various system operations without significant delays. For larger hospitals with multiple users accessing the application simultaneously, 8 GB RAM or higher is recommended to achieve better performance and stability.

Storage capacity is required for storing application files, JSON data files, reports, logs, and other system-related information. A minimum of 10 GB of free disk space is sufficient for development and small-scale deployment. However, hospitals handling large volumes of patient and dietary data may require additional storage capacity to maintain records and backups over extended periods.

A display device with a minimum screen resolution of  $1366 \times 768$  pixels is recommended for proper viewing of dashboards, reports, forms, and data tables. Higher-resolution monitors provide a better user experience by displaying more information clearly and reducing scrolling during data entry and management tasks.

The system also requires a stable network connection for communication between the client and server. Since HIMPS is a web-based application, users access the system through a browser connected to the local network or internet. Reliable network connectivity ensures smooth data transmission, real-time updates, and uninterrupted access to application features.

Input devices such as a keyboard and mouse are necessary for entering patient information, creating meal plans, updating records, and navigating the application. Additionally, hospitals may use printers for generating hard copies of reports and exported documents when required.

Overall, the hardware requirements of HIMPS are minimal and cost-effective, making the system suitable for deployment in healthcare institutions of various sizes. The application is designed to function efficiently on standard computing infrastructure while providing reliable performance for hospital nutrition and meal management operations.

## 2.2 Software Requirements

The Hospital Inpatient Meal Planning System (HIMPS) is developed using modern web technologies that provide a reliable, scalable, and user-friendly environment for managing hospital nutrition and dietary operations. The software requirements include the operating system, development tools, programming languages, runtime environments, web browsers, and supporting software necessary for the development, testing, deployment, and execution of the application. These software components work together to ensure smooth system performance, efficient data management, and an enhanced user experience.

The system can be developed and executed on modern operating systems such as **Windows 10, Windows 11, Linux, or macOS**. For this project, Windows 10 or Windows 11 is recommended because of its compatibility with development tools and ease of configuration. The operating system provides the environment required to run the application server, development tools, and web browsers used for testing and deployment.

A key software requirement for the backend is **Node.js**, which serves as the runtime environment for executing JavaScript code outside the browser. Node.js enables the development of scalable and efficient server-side applications. The Hospital Inpatient Meal Planning System uses Node.js to handle server operations, API requests, authentication processes, and data management tasks. A version of Node.js 18 or above is recommended to ensure compatibility with modern development practices and security updates.

The application also uses **Express.js**, a lightweight web application framework for Node.js. Express.js simplifies the creation of RESTful APIs, routing mechanisms, middleware management, and request handling. It provides a structured approach to backend development and improves maintainability of the system.

For frontend development, the system utilizes **HTML5**, **CSS3**, and **JavaScript ES6+**. HTML5 is used to design the structure and content of web pages, CSS3 is used for styling and responsive layouts, and JavaScript is used to implement dynamic functionality, form validation, data processing, and interactive user interfaces. These technologies collectively create a responsive and user-friendly environment for healthcare professionals.

The development process requires an Integrated Development Environment (IDE) such as **Visual Studio Code (VS Code)**. VS Code provides features including syntax highlighting, code completion, debugging tools, extensions, and project management capabilities that improve developer productivity. It is widely used for web development and supports all technologies utilized in the project.

A modern web browser is required to access and test the application. Browsers such as **Google Chrome**, **Microsoft Edge**, **Mozilla Firefox**, and **Safari** can be used. Google Chrome is recommended because of its developer tools, performance, and compatibility with modern web standards.

The project also requires **npm (Node Package Manager)**, which is installed along with Node.js. npm is used to install and manage project dependencies such as Express.js, CORS, and Nodemon. It simplifies package management and ensures that all required libraries are available for the application.

For API testing and debugging, **Postman** is used. Postman allows developers to test API endpoints, verify request and response data, and ensure proper communication between the frontend and backend components of the system.

Additionally, **Git** and **GitHub** are used for version control and source code management. These tools help developers track code changes, collaborate effectively, maintain backups, and manage project versions throughout the development lifecycle.

Overall, the software requirements of HIMPS provide a complete development and execution environment that supports efficient system implementation, testing, maintenance, and future scalability. These technologies ensure that the application remains reliable, secure, and easy to use within hospital nutrition management environments.

## 2.3 Development Tools and Frameworks

The development of the **Hospital Inpatient Meal Planning System (HIMPS)** requires a combination of modern tools, frameworks, and technologies that support efficient application design, coding, testing, and deployment. These tools help developers build a reliable, user-friendly, and scalable healthcare management system while ensuring ease of maintenance and future enhancements.

One of the primary development tools used in this project is **Visual Studio Code (VS Code)**. VS Code is a lightweight yet powerful Integrated Development Environment (IDE) that provides features such as syntax highlighting, code auto-completion, debugging support, and extension integration. It enables developers to write and manage HTML, CSS, JavaScript, and Node.js code efficiently. The built-in terminal and Git integration further simplify project management and version control activities.

The frontend of HIMPS is developed using **HTML5, CSS3, and JavaScript ES6+**. HTML5 is used to create the structure and content of web pages, including forms, tables, dashboards, and navigation menus. CSS3 is responsible for designing the user interface, implementing responsive layouts, and improving the overall visual appearance of the application. JavaScript provides dynamic functionality such as form validation, event handling, data processing, and interactive user experiences. Together, these technologies create a responsive and user-friendly web application.

For backend development, the project uses **Node.js**, which is a server-side JavaScript runtime environment. Node.js allows developers to execute JavaScript code outside the browser and supports efficient handling of multiple user requests. It is known for its speed, scalability, and event-driven architecture, making it suitable for web-based healthcare applications.

The project also utilizes **Express.js**, a lightweight and flexible web application framework built on top of Node.js. Express.js simplifies backend development by providing features such as

routing, middleware support, API creation, and request-response handling. It helps developers build RESTful services efficiently while maintaining clean and organized code structures.

To manage project dependencies and external libraries, **Node Package Manager (npm)** is used. npm simplifies the installation, updating, and maintenance of packages required for the application. Libraries such as Express.js, CORS, and Nodemon are managed using npm.

For testing and validating API endpoints, **Postman** is used. Postman enables developers to send HTTP requests, inspect responses, and verify backend functionality before integrating APIs with the frontend interface. This helps ensure system reliability and error-free communication between components.

Additionally, **Git** and **GitHub** are used for version control and project collaboration. Git tracks code changes and maintains project history, while GitHub provides a cloud-based repository for source code storage and team collaboration.

Overall, these development tools and frameworks provide a complete environment for designing, developing, testing, and maintaining the Hospital Inpatient Meal Planning System, ensuring efficient project execution and high-quality software development.

## CHAPTER-3 TECHNOLOGY STACK

### 3.1 Front-End Design

The front-end design of the **Hospital Inpatient Meal Planning System (HIMPS)** is developed to provide a simple, responsive, and user-friendly interface for hospital staff involved in nutrition and dietary management. The system is designed using **HTML5, CSS3, and JavaScript**, ensuring compatibility across modern web browsers and devices. The primary goal of the front-end design is to enable users to perform tasks efficiently while maintaining a clean and professional healthcare-oriented appearance.

The application follows a **dashboard-based design structure**, where users can access various modules through a navigation sidebar and top menu. The dashboard serves as the central control panel, displaying key information such as patient statistics, meal planning activities, kitchen order status, approval requests, and notifications. This allows users to quickly monitor hospital nutrition operations and access important functions without navigating through multiple pages.

The interface includes dedicated pages for patient management, dietary profile management, meal planning, allergy checking, calorie calculation, kitchen orders, approvals, notifications, reports, and administration. Each page is designed with well-structured forms, tables, cards, and buttons to improve usability and data organization. Search and filter options are also provided to help users locate records quickly and efficiently.

CSS3 is used to create responsive layouts using modern design techniques such as Flexbox and Grid systems. The design adapts to different screen sizes and resolutions, ensuring a consistent user experience across desktops and laptops. Color schemes, icons, typography, and visual elements are carefully selected to maintain readability and provide a professional healthcare environment.

JavaScript enhances the front-end by adding dynamic functionality such as form validation, real-time calculations, interactive tables, notifications, modal windows, and data updates. These interactive features improve user experience by reducing manual effort and providing immediate feedback during operations.

Overall, the front-end design of HIMPS focuses on simplicity, accessibility, and efficiency. By combining an intuitive user interface with responsive design principles and interactive functionality, the system enables administrators, dietitians, nurses, and kitchen staff to perform their responsibilities effectively while ensuring smooth navigation and improved productivity throughout the hospital nutrition management process.

## 3.2 Back-End Design

The back-end design of the **Hospital Inpatient Meal Planning System (HIMPS)** is responsible for managing business logic, processing user requests, handling data storage, and ensuring smooth communication between the user interface and the database. The system is developed using **Node.js** and **Express.js**, which provide a fast, scalable, and efficient environment for server-side application development.

Node.js serves as the runtime environment that executes server-side JavaScript code and handles multiple user requests simultaneously. Its event-driven architecture allows the application to process operations efficiently, making it suitable for healthcare systems where multiple users may access the platform at the same time. Express.js is used as the web application framework to simplify routing, middleware management, API creation, and request handling.

The backend follows a modular architecture where different functionalities are organized into separate components such as authentication, patient management, dietary profiles, meal management, meal planning, kitchen orders, notifications, and reporting. This structure improves maintainability, readability, and future scalability of the application.

The system uses **RESTful APIs** to facilitate communication between the frontend and backend. These APIs allow users to perform operations such as login authentication, patient registration, dietary profile creation, meal plan generation, allergy checking, kitchen order tracking, and report generation. Each request is processed by the server, which validates the data and returns the appropriate response.

For data storage, the current version of HIMPS uses **JSON files and browser localStorage** to store patient records, meal information, dietary profiles, kitchen orders, notifications, and user details. This approach simplifies development and testing while providing a lightweight storage solution for the project.

The backend also implements **Role-Based Access Control (RBAC)** to ensure that users can access only the features relevant to their roles. Administrators, dietitians, nurses, and kitchen staff have different permissions, improving security and operational control.

Overall, the back-end design of HIMPS provides a reliable foundation for managing hospital nutrition operations, ensuring secure data handling, efficient processing, and seamless integration between various modules of the system.

### 3.3 Database Design

The database design of the **Hospital Inpatient Meal Planning System (HIMPS)** is structured to efficiently store, manage, and retrieve information related to patients, dietary profiles, meals, meal plans, kitchen orders, users, notifications, and system history. The system currently uses **JSON files and browser localStorage** as its data storage mechanism, providing a lightweight and easy-to-maintain solution for managing application data. The database structure is designed to support smooth communication between different modules while maintaining data consistency and integrity.

The **User** entity stores information about system users such as administrators, dietitians, nurses, and kitchen staff. It includes attributes such as user ID, name, email address, password, role, and department. This information is used for authentication and role-based access control.

The **Patient** entity maintains patient-related information including patient ID, name, age, gender, ward, bed number, medical condition, doctor assigned, admission date, and status. Each patient record serves as the foundation for dietary planning and meal management activities.

The **Dietary Profile** entity contains nutrition-related information for each patient. It stores diet type, calorie requirements, protein targets, carbohydrate targets, fat requirements, allergies, dietary restrictions, water intake recommendations, and additional notes. This entity is linked to the patient entity through the patient ID.

The **Meal** entity stores information about available food items, including meal ID, meal name, meal type, calorie content, nutritional values, diet compatibility, and allergen details. These records are used when creating patient meal plans.

The **Meal Plan** entity manages daily meal schedules for patients. It contains patient details, selected meals, planning date, total calorie count, and approval status. Approved meal plans generate kitchen orders automatically.

The **Kitchen Order**, **Notification**, and **History** entities manage meal preparation tracking, system alerts, and audit records respectively. These entities ensure proper workflow monitoring and accountability.

Overall, the database design provides a well-organized structure that supports patient nutrition management, improves data accessibility, reduces redundancy, and enables efficient operation of all modules within the Hospital Inpatient Meal Planning System.

### 3.4 Version Control

Version control is an essential part of modern software development, as it helps developers manage changes made to source code throughout the project lifecycle. In the development of the **Hospital Inpatient Meal Planning System (HIMPS)**, version control plays a crucial role in tracking modifications, maintaining code history, preventing data loss, and supporting collaboration among team members. It allows developers to monitor project progress and efficiently manage updates, bug fixes, and feature enhancements.

The project uses **Git** as the version control system. Git is a distributed version control tool that enables developers to record changes made to files and source code over time. Every modification is stored as a separate version, allowing developers to review previous changes whenever necessary. This feature is particularly useful when debugging issues, recovering lost code, or comparing different versions of the application.

One of the major advantages of Git is its support for **branching and merging**. Developers can create separate branches to work on new features, bug fixes, or experiments without affecting the main project code. Once testing is completed successfully, these branches can be merged into the main branch. This approach improves development efficiency and reduces the risk of introducing errors into the stable version of the system.

The project also utilizes **GitHub**, a cloud-based repository hosting platform that integrates with Git. GitHub provides a centralized location where the project's source code can be stored,

managed, and shared. It enables team members to collaborate effectively by allowing them to upload changes, review code, track issues, and maintain project documentation. GitHub also serves as a backup repository, ensuring that the source code remains secure and accessible even if local files are lost.

Version control provides several benefits to the HIMPS project, including improved collaboration, easier maintenance, better project organization, and reliable backup management. It allows developers to keep track of every change made during development and ensures that multiple team members can work on the project simultaneously without conflicts.

Overall, Git and GitHub provide a structured and efficient version control environment for the Hospital Inpatient Meal Planning System. Their use helps maintain code quality, supports teamwork, simplifies project management, and ensures the long-term maintainability of the application.

### 3.5 APIs or External Services

Application Programming Interfaces (APIs) and external services play an important role in enabling communication between different components of the **Hospital Inpatient Meal Planning System (HIMPS)**. APIs allow the frontend and backend modules to exchange data efficiently, while external services enhance the functionality and user experience of the application. The system follows a RESTful architecture where the frontend sends requests to the backend server, and the server processes those requests and returns appropriate responses.

The backend of HIMPS is developed using **Node.js** and **Express.js**, which provide a framework for creating REST APIs. These APIs are responsible for handling various operations such as user authentication, patient management, dietary profile maintenance, meal management, meal planning, kitchen order tracking, notifications, and report generation. Through these APIs, users can perform Create, Read, Update, and Delete (CRUD) operations on system data without directly interacting with the database.

Some of the major APIs implemented in the system include:

- **Authentication API** – Validates user login credentials and manages user sessions.
- **Patient Management API** – Handles patient registration, updates, retrieval, and deletion of patient records.

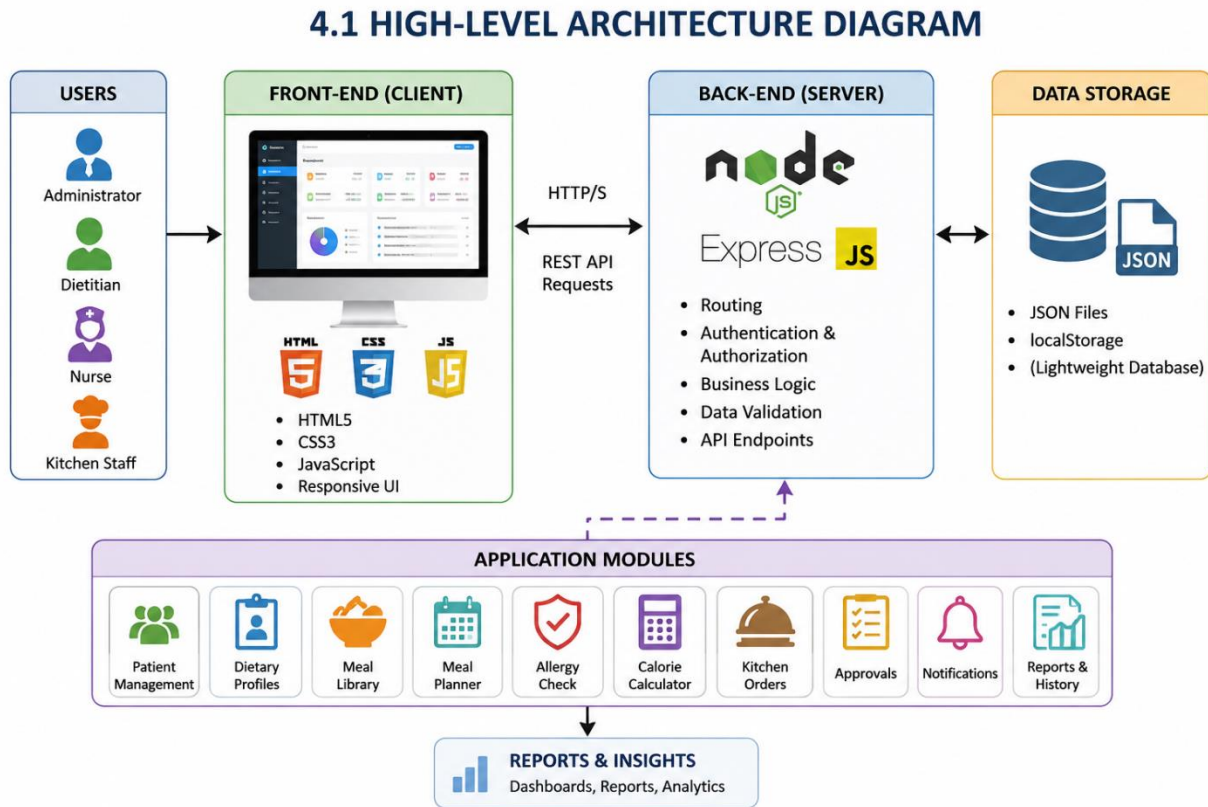
- **Dietary Profile API** – Manages patient diet types, allergies, restrictions, and nutritional requirements.
- **Meal Management API** – Maintains meal information, nutritional values, and diet compatibility details.
- **Meal Planning API** – Creates and manages daily meal plans for patients.
- **Kitchen Order API** – Tracks meal preparation and delivery status.
- **Notification API** – Generates alerts related to approvals, allergies, and system activities.
- **Dashboard API** – Provides summary statistics and information for the dashboard interface.

In addition to APIs, the project utilizes a few external services and libraries. **Google Fonts** is used to enhance the visual appearance of the user interface through modern typography. **CORS (Cross-Origin Resource Sharing)** middleware is implemented to enable secure communication between the frontend and backend components. The application also uses browser **localStorage** for temporary client-side data management and offline functionality.

Overall, APIs and external services form the backbone of communication within HIMPS. They ensure smooth data exchange, support modular development, improve system functionality, and contribute to the efficient operation of the Hospital Inpatient Meal Planning System.

## CHAPTER -4 SYSTEM ARCHITECTURE

### 4.1 High-level architecture diagram



### 4.2 Description of Each Layer (Frontend, Backend, Database)

The Hospital Inpatient Meal Planning System (HIMPS) follows a layered architecture that separates the application into three major layers: the **Frontend Layer**, **Backend Layer**, and **Database Layer**. This architecture improves system organization, maintainability, scalability, and performance. Each layer has a specific responsibility and works together to ensure smooth operation of the application.

#### Frontend Layer

The Frontend Layer is the presentation layer of the system and serves as the primary interface between users and the application. It is developed using **HTML5**, **CSS3**, and **JavaScript** and is responsible for displaying information and collecting user input. The frontend provides a responsive and user-friendly interface that allows administrators, dietitians, nurses, and kitchen

staff to interact with the system efficiently.

This layer includes various modules such as Login, Dashboard, Patient Management, Dietary Profiles, Meal Library, Meal Planner, Allergy Check, Calorie Calculator, Kitchen Orders, Notifications, Reports, and Admin Panel. Users can perform operations such as adding patient details, creating meal plans, checking allergies, and monitoring kitchen orders through graphical forms, tables, cards, and dashboards. JavaScript is used to provide dynamic functionality, form validation, and real-time updates, while CSS ensures a professional and responsive design across different devices.

### **Backend Layer**

The Backend Layer acts as the core processing unit of the application. It is developed using **Node.js and Express.js** and handles all business logic, request processing, authentication, and data management activities. This layer receives requests from the frontend, processes them according to system rules, and returns appropriate responses.

The backend manages functionalities such as user authentication, patient registration, dietary profile management, meal planning, kitchen order tracking, approval workflows, notifications, and report generation. It also implements Role-Based Access Control (RBAC), ensuring that each user can access only the features permitted for their role. RESTful APIs are used to establish communication between the frontend and backend, allowing seamless data exchange and system integration.

### **Database Layer**

The Database Layer is responsible for storing and managing all application data. In the current implementation, HIMPS uses **JSON files and browser localStorage** as storage mechanisms. This layer stores patient records, dietary profiles, meals, meal plans, kitchen orders, notifications, user accounts, and audit history.

The database layer ensures data persistence, retrieval, and consistency throughout the application. Whenever users create, update, delete, or retrieve information, the backend interacts with the database layer to perform the required operations. The stored data can also be used for generating reports, tracking system activities, and supporting future decision-making processes.

Together, these three layers create a structured and efficient architecture that enables HIMPS to provide reliable, secure, and user-friendly hospital nutrition management services.

### 4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

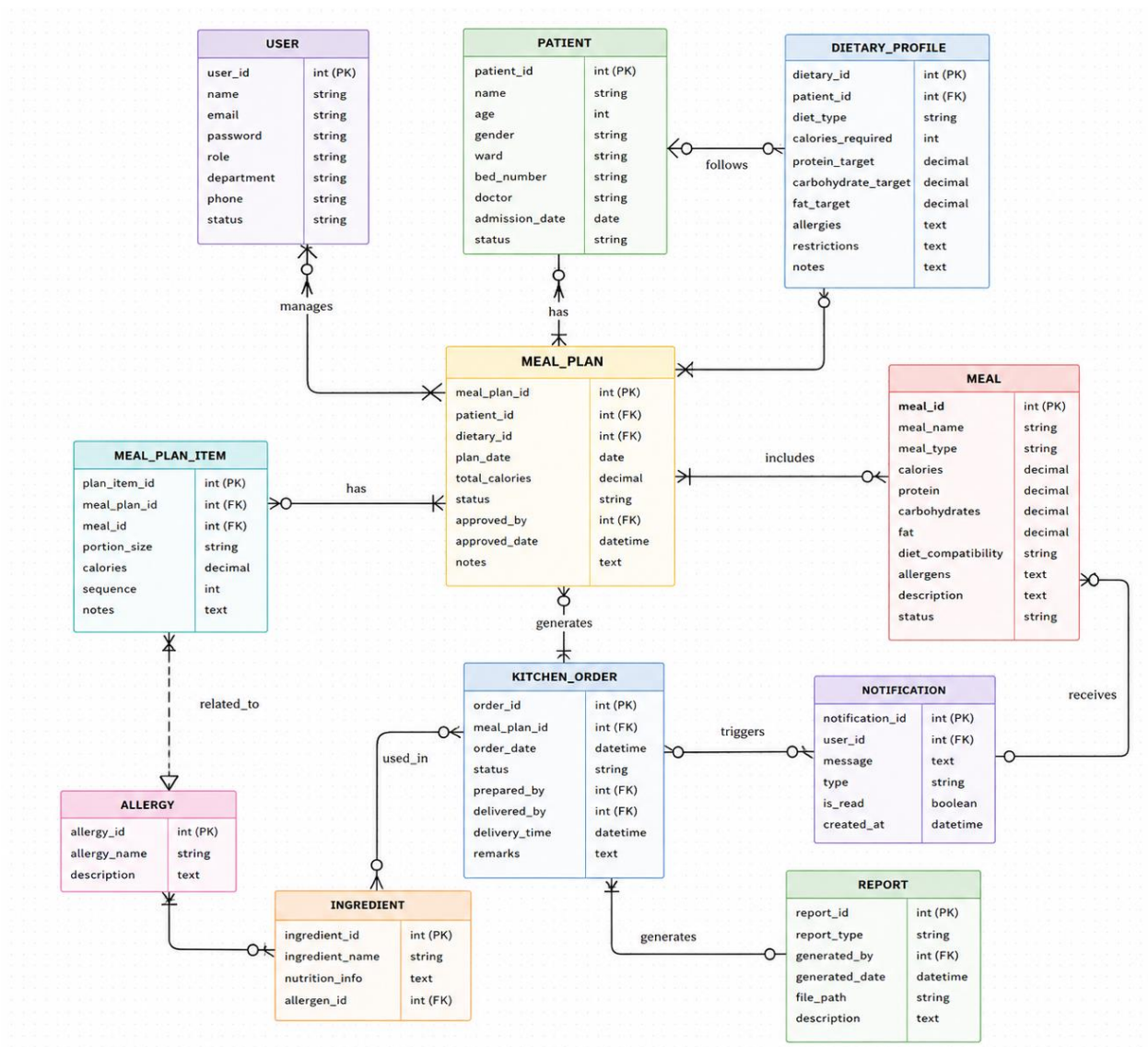
The **Hospital Inpatient Meal Planning System (HIMPS)** is designed to support both local and cloud-based deployment environments. During development and testing, the application is deployed locally using **Node.js** and **Express.js** servers running on a developer's machine. Users can access the system through a web browser connected to the local server, making development, debugging, and testing convenient.

For production environments, the application can be deployed on cloud platforms such as **AWS**, **Microsoft Azure**, **Google Cloud Platform (GCP)**, **Render**, **Railway**, or **Vercel**. Cloud deployment provides advantages such as remote accessibility, scalability, automatic backups, improved security, and higher availability. The frontend can be hosted separately while the backend services run on cloud servers, ensuring efficient resource utilization.

The project can also incorporate a **Continuous Integration and Continuous Deployment (CI/CD)** pipeline to automate software delivery. Tools such as **GitHub Actions**, **Jenkins**, or **GitLab CI/CD** can be used to automatically build, test, and deploy the application whenever changes are pushed to the repository. This reduces manual deployment effort, improves code quality, and enables faster delivery of updates.

Overall, the deployment architecture of HIMPS supports flexible hosting options, ensuring reliable performance, easier maintenance, and future scalability for hospital nutrition management operations.

## CHAPTER -5 DESIGN



## CHAPTER -6 IMPLEMENTATION

### 6.1 – Module-Wise Implementation

The Hospital Inpatient Meal Planning System (HIMPS) is developed using a modular architecture in which each module performs a specific function while working together to provide a complete hospital nutrition management solution. The modular approach improves maintainability, scalability, and overall system efficiency. The major modules implemented in the system include Authentication, Patient Management, Dietary Profile Management, Meal Management, Meal Planning, Allergy Checking, Calorie Calculation, Kitchen Order Management, Notifications, Reporting, and Administration.

#### Authentication and User Management Module

The Authentication Module provides secure access to the application through a login mechanism. Users are authenticated using their registered email addresses and passwords. The system supports Role-Based Access Control (RBAC), allowing different levels of access for Administrators, Dietitians, Nurses, and Kitchen Staff. After successful login, users are redirected to their respective dashboards where they can access authorized functionalities. This module ensures data security and prevents unauthorized access to sensitive patient information.

#### Patient Management Module

The Patient Management Module is responsible for maintaining patient records. Healthcare staff can add, update, search, and manage patient information such as patient ID, name, age, gender, ward, bed number, medical condition, assigned doctor, and admission date. The module acts as the foundation for all dietary planning activities since meal plans and dietary profiles are linked to patient records. Search and filtering options are provided to quickly locate patient information.

#### Dietary Profile Management Module

This module enables dietitians to create and maintain detailed dietary profiles for patients. Information such as diet type, calorie requirements, protein targets, carbohydrate requirements, fat limits, allergies, dietary restrictions, water intake recommendations, and nutritional notes can be stored. These profiles help healthcare professionals create personalized nutrition plans based on patient-specific medical needs and treatment objectives.

### **Meal Library Management Module**

The Meal Library Module stores information about available meals and food items. Each meal record contains details such as meal name, meal category, calorie content, nutritional values, suitable diet types, and allergen information. The centralized meal database allows dietitians to select appropriate meals while creating meal plans. This module ensures consistency in nutritional information and simplifies meal selection.

### **Meal Planning Module**

The Meal Planning Module allows dietitians to create daily meal schedules for patients. Meals can be assigned for breakfast, lunch, dinner, and snack sessions based on dietary requirements. The system automatically calculates total calorie values and compares them with the patient's nutritional targets. Meal plans are stored and can be reviewed or modified whenever necessary. This module significantly reduces manual effort and improves planning accuracy.

### **Allergy Check Module**

Patient safety is enhanced through the Allergy Check Module. Before assigning meals, the system compares patient allergy information with meal allergen details. If a conflict is detected, the system generates a warning message, preventing the assignment of unsafe meals. This automated verification process minimizes human errors and helps protect patients from allergic reactions and dietary complications.

### **Calorie Calculator Module**

The Calorie Calculator Module assists dietitians in determining patient nutritional requirements. It calculates calorie needs using factors such as age, gender, weight, height, activity level, and medical condition. The calculated values help create balanced meal plans and ensure that patients receive appropriate nutritional intake to support recovery and treatment.

### **Kitchen Order Management Module**

Once meal plans are approved, the Kitchen Order Module automatically generates food preparation requests. Kitchen staff can view assigned orders and update their status as Pending, Preparing, or Delivered. This module improves communication between departments and helps ensure timely meal preparation and delivery to patients.

## Notifications, Reports, and Administration Module

The Notification Module provides alerts related to approvals, allergy warnings, and important system events. The Reporting Module generates downloadable reports for patients, meal plans, dietary profiles, and kitchen operations. The Administration Module allows administrators to manage users, monitor system activities, review audit history, configure settings, and oversee overall application performance.

Together, these modules form an integrated hospital nutrition management system that improves efficiency, patient safety, communication, and operational control within healthcare organizations.

### 6.2 – Front-End Logic (UI Rendering and State Management)

The front-end logic of the **Hospital Inpatient Meal Planning System (HIMPS)** is responsible for managing user interactions, rendering interface components, handling data presentation, and maintaining application state. The system is developed using **HTML5, CSS3, and JavaScript**, which together provide a responsive and interactive user experience. The front-end serves as the bridge between users and the backend services, allowing healthcare professionals to perform various operations efficiently.

#### UI Rendering

User Interface (UI) rendering refers to the process of displaying data and application components dynamically on web pages. In HIMPS, JavaScript is used to generate and update content based on user actions and available data. Information such as patient records, dietary profiles, meal plans, kitchen orders, notifications, and reports is displayed using tables, forms, cards, and dashboard widgets.

When users access a module, the system retrieves the required data and renders it on the screen without requiring complex manual updates. For example, the Patient Management page dynamically displays patient details, while the Dashboard page presents statistics, alerts, and summary information. Modal windows and interactive forms are used to simplify data entry and editing operations. The use of responsive layouts ensures that content is displayed properly across different screen sizes and devices.

JavaScript event handlers are implemented to respond to user actions such as button clicks, form submissions, searches, filtering, and navigation. These events trigger appropriate functions that update the interface and provide immediate feedback to users.

## State Management

State management refers to the process of maintaining and updating application data during user interactions. HIMPS uses browser **localStorage** and JavaScript objects to manage application state. Data such as user sessions, patient information, dietary profiles, meal plans, notifications, and user preferences are temporarily stored and accessed whenever required.

When a user logs into the system, session information is stored and maintained throughout the active session. Similarly, when records are added, modified, or deleted, the state is updated immediately to reflect changes in the user interface. This ensures consistency between displayed information and underlying data.

The state management mechanism also supports dynamic features such as real-time calorie calculations, allergy conflict detection, kitchen order tracking, and notification updates. Changes made in one module can be reflected in related modules without requiring page reloads, improving usability and system responsiveness.

Overall, the front-end logic of HIMPS combines efficient UI rendering and effective state management to deliver a smooth, interactive, and user-friendly experience. These functionalities help healthcare professionals access information quickly, perform tasks accurately, and manage hospital nutrition operations efficiently.

## 6.3 API Integration

API (Application Programming Interface) Integration is a crucial part of the **Hospital Inpatient Meal Planning System (HIMPS)** as it enables seamless communication between the frontend and backend components of the application. The system follows a **RESTful API architecture**, where the frontend sends requests to the backend server, and the server processes those requests before returning the appropriate responses. This approach ensures efficient data exchange, modular development, and smooth system operation.

The backend of HIMPS is developed using **Node.js** and **Express.js**, which provide a robust environment for creating and managing APIs. These APIs act as intermediaries between the user interface and the database layer. Whenever users perform operations such as logging in, adding patient records, creating dietary profiles, generating meal plans, or updating kitchen orders, the frontend communicates with the backend through API calls.

One of the most important APIs implemented in the system is the **Authentication API**. This API verifies user credentials during login and ensures that only authorized users can access the application. Upon successful authentication, user details and session information are returned to the frontend, allowing role-based access to system features.

The **Patient Management API** handles operations related to patient records. It allows healthcare staff to create, retrieve, update, and manage patient information such as personal details, medical conditions, ward assignments, and admission records. Similarly, the **Dietary Profile API** manages nutritional information, allergies, dietary restrictions, calorie requirements, and other patient-specific dietary data.

The system also includes a **Meal Management API**, which stores and retrieves meal information including nutritional values, calorie content, allergens, and diet compatibility. This API supports the Meal Planner Module by providing the necessary meal data for creating customized meal plans. The **Meal Planning API** enables users to create, update, and monitor patient meal schedules while calculating nutritional totals automatically.

Additional APIs support modules such as **Kitchen Order Management**, **Notifications**, **Approvals**, and **Reporting**. These APIs allow the system to track meal preparation status, generate alerts, process approval workflows, and create downloadable reports. Communication between modules is achieved through API requests and responses, ensuring consistency across the entire application.

The APIs are tested using **Postman**, which helps verify request handling, response accuracy, and error management before deployment. This testing process ensures reliable communication between the frontend and backend components.

Overall, API integration plays a vital role in HIMPS by enabling secure, efficient, and real-time interaction between different system modules. It improves data accessibility, enhances user experience, and ensures smooth coordination of hospital nutrition management activities.

## 6.4 Authentication and Authorisation

Authentication and Authorization are essential security mechanisms implemented in the **Hospital Inpatient Meal Planning System (HIMPS)** to ensure that only authorized users can access the system and perform specific operations. Since the application handles sensitive patient information, dietary records, meal plans, and hospital management data, proper security controls are necessary to protect data confidentiality, integrity, and privacy.

### Authentication

Authentication is the process of verifying the identity of a user before granting access to the system. In HIMPS, users are required to log in using their registered email address and password. When a user submits login credentials, the system validates the entered information against stored user records. If the credentials are correct, access is granted and the user is redirected to the appropriate dashboard. If the credentials are invalid, the system displays an error message and denies access.

The authentication process ensures that only legitimate users can enter the application. User details such as email, password, role, and department are maintained within the system database. Once a user successfully logs in, session information is stored and maintained throughout the active session, allowing users to navigate different modules without repeatedly entering credentials.

The login module also improves security by preventing unauthorized individuals from accessing patient records, dietary information, kitchen orders, and administrative functions. Authentication serves as the first line of defense against unauthorized system usage.

### Authorization

Authorization is the process of determining what actions an authenticated user is allowed to perform within the system. HIMPS implements **Role-Based Access Control (RBAC)** to manage user permissions efficiently. Different users are assigned different roles based on their responsibilities within the hospital.

The system supports the following user roles:

- **Administrator**
- **Dietitian**
- **Nurse**
- **Kitchen Staff**

Each role has access only to the modules and functions relevant to its responsibilities. Administrators have full system access, including user management, approvals, reports, and system settings. Dietitians can manage dietary profiles, calorie calculations, meal planning, and allergy checks. Nurses can view patient dietary information and monitor meal plans, while kitchen staff can access kitchen orders and update meal preparation and delivery status.

Role-based authorization improves system security by restricting sensitive operations to authorized personnel only. It also prevents accidental modifications to critical data and ensures accountability within the hospital environment.

Overall, the Authentication and Authorization mechanisms implemented in HIMPS provide a secure framework for user access management. These features protect patient information, support role-specific functionality, enhance system reliability, and ensure that hospital nutrition management operations are performed in a controlled and secure manner.

## CHAPTER 7- FEATURES

### 7.1 List of Main Features

The **Hospital Inpatient Meal Planning System (HIMPS)** is designed to provide a complete solution for managing patient nutrition, dietary planning, and hospital food service operations. The system integrates multiple modules that work together to improve efficiency, reduce manual work, and ensure patient safety. The following are the major features implemented in the application.

#### 1. User Authentication and Role-Based Access

The system provides a secure login mechanism for different categories of users, including Administrators, Dietitians, Nurses, and Kitchen Staff. Role-Based Access Control (RBAC) ensures that users can access only the modules and functionalities relevant to their responsibilities.

#### 2. Dashboard Management

The dashboard serves as the central interface of the system and displays important information such as active patients, pending approvals, kitchen order status, notifications, and overall system statistics. It provides users with a quick overview of daily operations.

#### 3. Patient Management

This feature allows healthcare staff to add, edit, view, search, and manage patient records. Information such as patient details, medical conditions, ward assignments, bed numbers, and admission dates can be maintained efficiently.

#### 4. Dietary Profile Management

Dietitians can create and manage patient-specific dietary profiles. The module stores information such as diet type, calorie requirements, allergies, dietary restrictions, nutritional targets, water intake recommendations, and additional dietary instructions.

## **5. Meal Library Management**

The system maintains a centralized meal library containing meal names, nutritional values, calorie content, diet compatibility, and allergen information. This simplifies meal selection and planning processes.

## **6. Meal Planning System**

Dietitians can create daily meal plans by selecting suitable meals for breakfast, lunch, dinner, and snacks. The system helps ensure that nutritional requirements are met while maintaining dietary restrictions.

## **7. Allergy Checking**

An automated allergy detection feature compares patient allergies with meal ingredients and generates alerts whenever a conflict is detected. This improves patient safety and prevents dietary errors.

## **8. Calorie Calculator**

The calorie calculator estimates daily nutritional requirements based on patient age, weight, height, gender, activity level, and medical condition. This helps dietitians prepare balanced meal plans.

## **9. Kitchen Order Tracking**

Approved meal plans automatically generate kitchen orders. Kitchen staff can track meal preparation and update order status as Pending, Preparing, or Delivered.

## **10. Notifications, Reports, and Audit History**

The system provides notifications for approvals, allergy alerts, and important events. It also generates reports and maintains an audit history of activities, ensuring transparency, accountability, and effective hospital nutrition management.

Together, these features make HIMPS a comprehensive and efficient solution for hospital inpatient meal planning and dietary management.

## 7.2 How Each Feature Works

The **Hospital Inpatient Meal Planning System (HIMPS)** consists of several integrated features that work together to manage patient nutrition and hospital meal operations efficiently. Each feature performs a specific function while interacting with other modules to ensure smooth workflow management.

The **User Authentication** feature verifies user credentials during login and grants access based on assigned roles such as Administrator, Dietitian, Nurse, or Kitchen Staff. This ensures secure access and prevents unauthorized usage of the system.

The **Dashboard** feature provides a centralized overview of hospital nutrition activities. It displays important information such as active patients, pending approvals, kitchen order status, notifications, and system statistics, enabling users to monitor operations quickly.

The **Patient Management** feature allows healthcare staff to register, update, search, and manage patient records. Patient information serves as the foundation for dietary planning and nutrition management activities.

The **Dietary Profile Management** feature stores patient-specific nutritional information, including diet type, calorie requirements, allergies, restrictions, and special dietary instructions. Dietitians use this information to prepare personalized meal plans.

The **Meal Library** feature maintains a collection of meals with nutritional values, calorie information, diet compatibility, and allergen details. This database helps users select appropriate meals while creating meal plans.

The **Meal Planning** feature enables dietitians to schedule meals for breakfast, lunch, dinner, and snacks according to patient dietary requirements. The system calculates nutritional totals and helps maintain balanced meal schedules.

The **Allergy Check** feature automatically compares patient allergies with meal allergens. If a conflict is detected, warning notifications are generated to prevent unsafe meal assignments.

The **Calorie Calculator** feature calculates daily calorie requirements based on patient age, weight, height, gender, activity level, and medical condition. The results assist dietitians in designing suitable nutrition plans.

The **Kitchen Order Management** feature converts approved meal plans into kitchen orders. Kitchen staff can update order status as Pending, Preparing, or Delivered to ensure proper meal tracking.

Finally, the **Notifications, Reports, and Audit History** features provide alerts, generate reports, and maintain records of system activities. These features improve communication, accountability, and decision-making while ensuring efficient hospital nutrition management.

## CHAPTER -8 TESTING

### 8.1 Postman Testing (Frontend/Backend)

Postman testing was performed to verify the functionality, reliability, and accuracy of the APIs implemented in the **Hospital Inpatient Meal Planning System (HIMPS)**. Postman is a widely used API testing tool that allows developers to send HTTP requests, analyze responses, validate data flow, and identify errors before deployment. In this project, Postman was used primarily for testing communication between the frontend and backend components of the application.

The backend of HIMPS is developed using **Node.js** and **Express.js**, where RESTful APIs handle operations such as authentication, patient management, dietary profile maintenance, meal management, meal planning, kitchen order tracking, and notifications. Postman was used to ensure that each API endpoint functioned correctly and returned the expected responses under different conditions.

The **Authentication API** was tested by sending login requests with valid and invalid user credentials. Successful login attempts returned user information and access permissions, while invalid credentials generated appropriate error messages. This verified the correctness of the authentication process and ensured secure access control.

The **Patient Management APIs** were tested using Create, Read, Update, and Delete (CRUD) operations. Patient records were added, modified, retrieved, and managed through API requests. The responses were checked to ensure that patient information was stored and retrieved correctly without data loss or duplication.

Similarly, the **Dietary Profile APIs** were tested to verify the creation and maintenance of patient dietary profiles. Information such as diet types, calorie requirements, allergies, and restrictions was submitted through API requests and validated against stored records.

The **Meal Management and Meal Planning APIs** were tested to ensure proper retrieval of meal information and successful generation of meal plans. Nutritional values, calorie totals, and dietary compatibility were verified during testing to confirm accurate data processing.

The **Kitchen Order APIs** were tested by updating meal preparation statuses and verifying order tracking functionality. Requests for changing order states from Pending to Preparing and Delivered were successfully validated.

Frontend testing was also performed by connecting the user interface with backend APIs and verifying that data displayed on pages matched the API responses. Forms, dashboards, tables, notifications, and reports were tested to ensure proper integration between frontend and backend components.

Overall, Postman testing played a vital role in identifying issues, validating API functionality, and ensuring reliable communication between system modules. The testing process improved application quality, reduced errors, and contributed to the successful implementation of the Hospital Inpatient Meal Planning System.

## 8.2 Integration Testing

Integration testing was conducted in the **Hospital Inpatient Meal Planning System (HIMPS)** to verify that all modules communicate and function correctly when combined as a complete system. While unit testing focuses on individual components, integration testing ensures that data flows accurately between interconnected modules and that system functionalities operate as expected in real-world scenarios. The primary objective of integration testing was to identify interface-related issues, data inconsistencies, and communication errors between different modules.

One of the most important integration tests performed was between the **Patient Management Module** and the **Dietary Profile Module**. When a new patient record is created, the patient information should be available for creating dietary profiles. The testing process confirmed that patient data was successfully transferred and displayed in the dietary profile section without any data mismatch or duplication.

Another critical integration test involved the **Dietary Profile Module** and the **Meal Planner Module**. The system was tested to ensure that patient dietary requirements, allergies, calorie targets, and restrictions were automatically considered while creating meal plans. The results confirmed that meal planning was accurately based on the stored dietary profile information.

The integration between the **Meal Library Module** and the **Meal Planner Module** was also tested. Meal information such as calorie values, nutritional content, diet compatibility, and allergen details was successfully retrieved from the meal library and used during meal plan creation. This ensured consistency and accuracy in nutritional planning.

Testing was also conducted between the **Meal Planner Module** and the **Allergy Check Module**. When meals were assigned to patients, the system automatically compared meal allergens with patient allergy records. The integration test verified that warning messages were generated whenever conflicts were detected, helping prevent unsafe meal assignments.

The interaction between the **Meal Planner Module** and the **Kitchen Order Module** was another major area of testing. Approved meal plans automatically generated kitchen orders, and these orders were successfully transferred to the kitchen management section. Kitchen staff could then update preparation and delivery statuses without affecting meal plan records.

Additional integration testing was performed for **Notifications, Approval Workflows, Dashboard Updates, and Reporting Modules**. System notifications were generated correctly, approval statuses were updated accurately, dashboard statistics reflected current data, and reports contained the latest information from all connected modules.

Overall, integration testing confirmed that all modules within HIMPS work together effectively and exchange data accurately. The testing process helped identify and eliminate communication issues, ensuring a stable, reliable, and efficient hospital nutrition management system.

### 8.3 Tools Used for Testing

Testing is a critical phase in the software development lifecycle, as it helps ensure that the application functions correctly, meets user requirements, and performs reliably under different conditions. During the development of the **Hospital Inpatient Meal Planning System (HIMPS)**, several testing tools were used to verify the functionality, performance, security, and integration of different system modules. These tools assisted in identifying errors, validating system behavior, and improving the overall quality of the application.

One of the primary tools used for testing was **Postman**. Postman is a widely used API testing platform that allows developers to send HTTP requests and analyze server responses. In HIMPS, Postman was used to test backend REST APIs related to user authentication, patient

management, dietary profiles, meal planning, kitchen orders, notifications, and reporting. Through Postman, developers verified request handling, response accuracy, status codes, and data validation. The tool helped ensure that communication between the frontend and backend components functioned correctly.

Another important testing tool used was **Google Chrome Developer Tools (DevTools)**. Chrome DevTools provides a collection of debugging and inspection features that assist developers in testing frontend functionality. It was used to inspect HTML elements, analyze CSS styles, monitor JavaScript execution, identify console errors, and evaluate network requests. The responsive design testing feature also helped verify that the application interface displayed correctly across different screen sizes and resolutions.

**Visual Studio Code (VS Code)** played a significant role during testing and debugging. The built-in debugging tools available in VS Code enabled developers to identify coding errors, monitor variable values, track execution flow, and resolve logic-related issues efficiently. Syntax checking and extension support further improved code quality during development and testing.

The system was also tested using various **web browsers**, including Google Chrome and Microsoft Edge. Cross-browser testing ensured that the application functioned consistently and maintained a uniform user experience regardless of the browser used by hospital staff.

Additionally, **Node.js runtime logs** were utilized to monitor backend operations and identify server-side errors during testing. These logs provided valuable information regarding API execution, request handling, and application performance.

Version control tools such as **Git and GitHub** indirectly supported testing by allowing developers to track code changes, revert faulty implementations, and maintain stable versions of the application during testing phases.

Overall, the use of Postman, Chrome DevTools, VS Code, Node.js logs, web browsers, and version control tools ensured thorough testing of HIMPS. These tools helped improve application reliability, functionality, performance, and user satisfaction while contributing to the successful implementation of the Hospital Inpatient Meal Planning System.

## 8.4 Test Cases and Results

Testing was conducted on the Hospital Inpatient Meal Planning System (HIMPS) to ensure that all modules functioned correctly and met the expected requirements. Various test cases were executed to verify system reliability, accuracy, and user interaction.

| Test Case ID | Test Scenario           | Input                                 | Expected Result                | Actual Result                  | Status |
|--------------|-------------------------|---------------------------------------|--------------------------------|--------------------------------|--------|
| TC01         | User Login              | Valid email and password              | User successfully logs in      | Login successful               | Pass   |
| TC02         | Invalid Login           | Incorrect credentials                 | Error message displayed        | Error message shown            | Pass   |
| TC03         | Add Patient             | Valid patient details                 | Patient record created         | Record created successfully    | Pass   |
| TC04         | Create Dietary Profile  | Valid dietary information             | Profile saved successfully     | Profile saved                  | Pass   |
| TC05         | Allergy Check           | Patient allergy matches meal allergen | Warning alert generated        | Alert displayed correctly      | Pass   |
| TC06         | Meal Planning           | Select meals for patient              | Meal plan generated            | Meal plan created              | Pass   |
| TC07         | Calorie Calculation     | Valid patient parameters              | Calories calculated accurately | Correct values generated       | Pass   |
| TC08         | Kitchen Order Update    | Change status to Delivered            | Order status updated           | Status updated successfully    | Pass   |
| TC09         | Notification Generation | Approval request created              | Notification displayed         | Notification generated         | Pass   |
| TC10         | Report Export           | Export report request                 | CSV report downloaded          | Report downloaded successfully | Pass   |

The testing results indicate that all major functionalities of the system performed as expected. No critical errors were identified during testing, confirming that the application is stable, reliable, and suitable for hospital nutrition management operations.

## CHAPTER -9 DEPLOYMENT

### 9.1 Steps to Deploy

Deploying the **Hospital Inpatient Meal Planning System (HIMPS)** involves setting up the required software environment, configuring the application, and launching both the frontend and backend components. Proper deployment ensures that the application functions correctly and can be accessed by authorized users within the hospital network or through cloud hosting platforms. The following steps describe the deployment process of the system.

#### Step 1: Install Required Software

The first step is to install all necessary software components required for running the application. This includes:

- Node.js (Version 18 or above)
- npm (Node Package Manager)
- Visual Studio Code (Optional)
- Google Chrome or any modern web browser
- Git (Optional for version control)

After installation, verify that Node.js and npm are correctly installed by executing the following commands in the command prompt:

```
node -v  
npm -v
```

#### Step 2: Obtain the Project Files

Download or clone the HIMPS project source code from the project repository. Ensure that all frontend and backend files are available in the project directory. The project should contain HTML pages, CSS files, JavaScript files, server files, configuration files, and required dependencies.

#### Step 3: Install Project Dependencies

Navigate to the project folder using the command prompt or terminal and install all required packages using npm:

```
npm install
```

This command downloads and installs all dependencies specified in the package configuration file, including Express.js, CORS, and other required libraries.

#### **Step 4: Configure the Application**

Review the project configuration files and ensure that server settings, file paths, and storage locations are correctly configured. Verify that JSON data files and application resources are accessible to the server.

#### **Step 5: Start the Backend Server**

Launch the Node.js server using the following command:

```
npm start
```

or

```
node server.js
```

Once started, the server begins listening for incoming requests and provides API services to the frontend application.

#### **Step 6: Access the Application**

Open a web browser and navigate to the application URL. For local deployment, users can access the system through:

```
http://localhost:3000
```

The login page will appear, allowing authorized users to access the system.

#### **Step 7: Verify System Functionality**

After deployment, perform testing to verify that all modules such as login, patient management, meal planning, dietary profiles, kitchen orders, notifications, and reporting functions are operating correctly.

By following these deployment steps, HIMPS can be successfully installed and executed in a local or production environment, ensuring reliable access to hospital nutrition management services.

## 9.2 Environment Configuration

Environment configuration is an important step in the deployment and execution of the **Hospital Inpatient Meal Planning System (HIMPS)**. It ensures that all required software components, dependencies, and system settings are properly configured before the application is launched. A well-configured environment improves system stability, performance, and compatibility across different deployment platforms.

The HIMPS application is developed using **HTML5, CSS3, JavaScript, Node.js, and Express.js**. Therefore, the first requirement is the installation of **Node.js** and **npm (Node Package Manager)**. Node.js provides the runtime environment required for executing server-side JavaScript, while npm manages the project dependencies and libraries. After installation, developers can verify the setup using version commands to ensure that the required software is available.

The project dependencies are installed through the package configuration file using the npm installation command. This process automatically downloads and configures all necessary libraries such as Express.js, CORS, and other supporting packages required for backend operations.

The server configuration includes defining the application port, enabling middleware, and configuring static file access. In the current implementation, the backend server is configured to run on **Port 3000**, allowing communication between the frontend interface and backend services. JSON files are used as the primary data storage mechanism, and appropriate file paths are configured to ensure proper data access and management.

The application also requires a compatible web browser such as **Google Chrome, Microsoft Edge, or Mozilla Firefox** for accessing the user interface. Browser settings should allow JavaScript execution and local storage access to ensure full functionality of the application.

For development and testing purposes, **Visual Studio Code (VS Code)** is used as the development environment, while **Postman** is used for API testing and validation. Additionally, Git and GitHub can be configured for source code management and version control.

Overall, proper environment configuration ensures that HIMPS operates efficiently by providing a stable platform for application execution, database access, API communication, and user interaction. It forms the foundation for successful deployment and reliable hospital nutrition management operations.

### 9.3 Hosting of Frontend and Backend

The **Hospital Inpatient Meal Planning System (HIMPS)** can be hosted using various deployment platforms depending on the scale and requirements of the healthcare organization. Since the application consists of both frontend and backend components, each part can be hosted separately or together on the same server. Proper hosting ensures that the application remains accessible, secure, and available to authorized users whenever required.

#### Frontend Hosting

The frontend of HIMPS is developed using **HTML5, CSS3, and JavaScript**, making it a static web application that can be hosted on several web hosting platforms. Popular options include **Vercel, Netlify, GitHub Pages**, and cloud storage services provided by AWS and Azure. These platforms provide fast content delivery, automatic deployment, and reliable uptime.

For deployment, the frontend files are uploaded to the hosting platform, which then generates a public URL that users can access through a web browser. Since the frontend contains only static resources such as HTML pages, CSS stylesheets, JavaScript files, and images, hosting requirements are minimal and cost-effective. The hosted frontend communicates with the backend server through API requests to retrieve and update data.

#### Backend Hosting

The backend of HIMPS is developed using **Node.js** and **Express.js**. It can be hosted on cloud platforms such as **Render, Railway, AWS EC2, Microsoft Azure, Google Cloud Platform (GCP)**, or **Heroku**. These platforms support Node.js applications and provide server environments for running backend services.

The backend server manages user authentication, patient records, dietary profiles, meal planning, notifications, kitchen orders, and report generation. During deployment, all required dependencies are installed, and the server is configured to run continuously. Environment variables and server configurations are also set to ensure secure and efficient operation.

### **Integrated Hosting Approach**

In small-scale deployments, both frontend and backend components can be hosted on the same server. For larger deployments, separating frontend and backend hosting improves scalability, security, and performance. Cloud-based hosting also provides additional benefits such as automatic backups, load balancing, monitoring, and high availability.

Overall, hosting the HIMPS frontend and backend on reliable cloud platforms ensures continuous access, better performance, enhanced security, and easier maintenance. It enables healthcare professionals to access the system from different locations while supporting efficient hospital nutrition and meal management operations.

## CHAPTER -10 CHALLENGES & LIMITATIONS

### 10.1 Issues Faced During Development

The development of the **Hospital Inpatient Meal Planning System (HIMPS)** involved several technical and implementation challenges. As the project integrates multiple modules such as patient management, dietary profiles, meal planning, allergy checking, kitchen order tracking, and reporting, various issues were encountered during different stages of development. Identifying and resolving these challenges played an important role in improving the functionality, reliability, and performance of the application.

One of the major challenges faced during development was **designing a structured data management system**. Since the application handles multiple interconnected entities such as patients, meals, dietary profiles, meal plans, and kitchen orders, maintaining relationships between these records required careful planning. Ensuring consistency of data across different modules was initially difficult, especially when updating or retrieving information from multiple sources.

Another significant issue was **implementing role-based access control (RBAC)**. The system supports different user roles, including administrators, dietitians, nurses, and kitchen staff. Each role requires access to specific modules and functionalities. Managing permissions and ensuring that users could only access authorized sections of the system required additional logic and testing. Incorrect permission settings could lead to unauthorized access or restricted functionality.

Developing the **allergy checking mechanism** was also challenging. The system needed to compare patient allergy information with meal allergen data accurately and generate warnings whenever conflicts were detected. Implementing this feature required careful handling of dietary data and extensive testing to ensure that no potential allergy conflicts were missed.

Another challenge involved **meal planning and calorie calculations**. Different patients require personalized nutritional plans based on their medical conditions, dietary restrictions, age, weight, and activity levels. Developing algorithms that accurately calculate calorie requirements and nutritional targets while supporting multiple diet types required considerable effort and validation.

The integration between frontend and backend components also presented difficulties. Ensuring smooth communication through REST APIs, handling data synchronization, and managing API responses required continuous debugging and testing. Issues such as incorrect data formatting, missing fields, and response errors occasionally affected system functionality during development.

Additionally, maintaining a responsive and user-friendly interface was challenging. The application contains multiple pages, tables, forms, dashboards, and reports that must function correctly across different screen sizes and browsers. Considerable effort was invested in improving usability, navigation, and interface consistency.

Testing and debugging were ongoing challenges throughout the project. Identifying logical errors, fixing integration issues, validating API functionality, and ensuring proper data flow between modules required extensive testing using Postman, browser developer tools, and manual verification methods.

Despite these challenges, all major issues were successfully resolved through careful planning, modular development practices, continuous testing, and systematic debugging. Overcoming these obstacles contributed significantly to the successful implementation of the Hospital Inpatient Meal Planning System.

## 10.2 Solutions Applied

During the development of the **Hospital Inpatient Meal Planning System (HIMPS)**, several challenges were encountered related to data management, system integration, security, and user interface design. To ensure successful implementation, appropriate solutions were applied to address these issues and improve the overall performance and reliability of the application.

One of the primary challenges involved managing relationships between multiple modules such as patients, dietary profiles, meals, meal plans, and kitchen orders. To solve this issue, a **modular architecture** was adopted. Each module was designed independently while maintaining proper connections through unique identifiers such as patient IDs and meal IDs. This approach improved data consistency and simplified information retrieval across the system.

To address security concerns and unauthorized access, **Role-Based Access Control (RBAC)** was implemented. Different user roles, including Administrator, Dietitian, Nurse, and Kitchen

Staff, were assigned specific permissions according to their responsibilities. Authentication mechanisms were integrated into the login system to verify user credentials before granting access. This solution enhanced system security and protected sensitive patient information.

The challenge of detecting dietary conflicts and food allergies was resolved by developing an **automated allergy checking mechanism**. The system compares patient allergy records with meal allergen information before meals are assigned. Whenever a conflict is detected, warning messages are generated automatically. This solution significantly reduces the possibility of dietary errors and improves patient safety.

To overcome difficulties in nutritional calculations, a dedicated **Calorie Calculator Module** was introduced. The module automatically calculates calorie requirements based on patient age, weight, height, gender, activity level, and medical condition. This eliminated manual calculations, reduced errors, and improved the accuracy of meal planning.

Integration issues between frontend and backend components were addressed through the use of **RESTful APIs**. Standardized API endpoints were developed to facilitate smooth communication between modules. Extensive testing using Postman ensured reliable data exchange and minimized communication errors.

User interface challenges were resolved by implementing a **responsive design** using HTML, CSS, and JavaScript. Modern layout techniques such as Flexbox and Grid were used to improve navigation, readability, and accessibility across different devices and screen resolutions.

Continuous testing, debugging, and code reviews were conducted throughout development to identify and resolve issues quickly. These solutions collectively improved system stability, functionality, security, and usability, contributing to the successful implementation of the Hospital Inpatient Meal Planning System.

### 10.3 Current Limitations or Known Bugs

Although the **Hospital Inpatient Meal Planning System (HIMPS)** successfully fulfills its primary objectives, there are still certain limitations and areas that can be improved in future versions. These limitations are mainly related to data storage, scalability, integration capabilities, and advanced healthcare functionalities.

One of the major limitations of the current system is the use of **JSON files and browser localStorage** for data storage. While this approach is suitable for academic projects and small-scale deployments, it may not be efficient for large hospitals that handle thousands of patient records and daily meal transactions. A dedicated database management system such as MySQL or PostgreSQL would provide better scalability, performance, and data security.

Another limitation is the lack of **real-time synchronization** between multiple users. Since the current implementation relies on local storage and file-based data management, simultaneous updates by different users may not be reflected instantly across all connected devices. This can lead to temporary inconsistencies in displayed information.

The system currently operates as a **web application only** and does not provide a dedicated mobile application for nurses, dietitians, or kitchen staff. As a result, users must access the system through a web browser, which may be less convenient in certain healthcare environments.

A known limitation is the absence of integration with **Electronic Health Record (EHR)** systems and other hospital management software. Patient information must be entered manually, which may increase administrative workload and create the possibility of duplicate data entry.

The notification system is currently limited to in-application alerts and does not support advanced communication methods such as email notifications, SMS alerts, or push notifications. Additionally, report generation is limited to basic export formats and does not include advanced analytics or graphical reporting features.

Some minor interface issues may occur when the application is accessed on smaller screens or lower-resolution devices, requiring further optimization for complete responsiveness.

Despite these limitations, the system remains stable and functional for hospital nutrition management. Future enhancements such as database integration, cloud deployment, mobile applications, real-time synchronization, and advanced reporting can significantly improve the overall performance, scalability, and usability of the Hospital Inpatient Meal Planning System.

## CHAPTER -11 FUTURE ENHANCEMENTS

### 11.1 Planned Features

The **Hospital Inpatient Meal Planning System (HIMPS)** has been designed as a foundation for digital nutrition management within hospitals. While the current version successfully manages patient dietary profiles, meal planning, allergy checking, calorie calculation, kitchen orders, and reporting, several advanced features are planned for future development. These enhancements aim to improve system functionality, scalability, user convenience, and overall healthcare service quality.

One of the major planned features is the integration of a **Relational Database Management System (RDBMS)** such as MySQL or PostgreSQL. The current system uses JSON files and localStorage for data storage, which is suitable for small-scale environments. However, implementing a database solution will improve data security, scalability, performance, and support for multiple users accessing the system simultaneously.

Another important enhancement is the implementation of **JWT (JSON Web Token) Authentication** and advanced security mechanisms. This feature will provide more secure user authentication, session management, and protection against unauthorized access. Additional security measures such as password encryption, multi-factor authentication, and role-based permissions can further strengthen system security.

The development of a **Mobile Application** is also planned. A mobile version of HIMPS would allow dietitians, nurses, and kitchen staff to access patient information, update meal statuses, and monitor dietary plans directly from smartphones and tablets. This would improve accessibility and operational efficiency within hospital environments.

Future versions of the system may include **Email and SMS Notification Services**. These services can automatically notify healthcare staff about meal plan approvals, allergy alerts, order updates, and important dietary changes. Real-time communication would help improve coordination among departments and reduce response times.

Another planned feature is **Artificial Intelligence (AI)-Based Diet Recommendations**. By analyzing patient medical conditions, dietary history, and nutritional requirements, the system

could suggest personalized meal plans and nutritional improvements automatically. This would assist dietitians in making more informed decisions and enhance patient care.

Integration with **Electronic Health Records (EHR)** and Hospital Management Systems is also proposed. Such integration would enable automatic retrieval of patient information, reducing manual data entry and improving data accuracy.

Additionally, advanced reporting and analytics features are planned to provide graphical dashboards, nutritional trend analysis, and performance monitoring. These reports would help hospital administrators evaluate service quality and make data-driven decisions.

Overall, these planned features will transform HIMPS into a more intelligent, scalable, secure, and comprehensive healthcare nutrition management solution capable of meeting the evolving needs of modern hospitals.

## 11.2 Possible Integrations or Optimisations

The **Hospital Inpatient Meal Planning System (HIMPS)** offers significant opportunities for future integrations and optimizations that can enhance its functionality, efficiency, and scalability. These improvements will help the system adapt to the evolving needs of healthcare organizations while providing better support for patient nutrition management.

One of the most important potential integrations is with **Electronic Health Record (EHR)** systems and Hospital Management Systems (HMS). Integrating HIMPS with existing healthcare databases would allow patient information, medical history, diagnoses, and treatment details to be automatically retrieved. This would eliminate repetitive data entry, reduce human errors, and improve the accuracy of dietary planning.

Another valuable integration is with **laboratory and diagnostic systems**. By accessing patient test results such as blood sugar levels, cholesterol values, kidney function reports, and nutritional assessments, the system could generate more accurate dietary recommendations based on real-time medical data. This would improve personalized nutrition planning and patient outcomes.

The system can also be optimized by implementing **cloud-based infrastructure**. Cloud deployment would provide better scalability, remote accessibility, automatic backups, and improved system availability. Hospitals with multiple branches could access the same centralized platform, ensuring consistent dietary management across locations.

Performance optimization can be achieved by replacing JSON-based storage with **relational database systems** such as MySQL or PostgreSQL. This would improve data retrieval speed, support large datasets, and enable efficient multi-user operations. Database indexing and query optimization techniques could further enhance system performance.

Another possible enhancement is the implementation of **Artificial Intelligence (AI) and Machine Learning (ML)** algorithms. These technologies could analyze patient data and automatically recommend suitable meal plans, identify nutritional risks, and predict dietary requirements based on medical conditions and historical records.

Additionally, integrating **email, SMS, and push notification services** would improve communication among healthcare professionals by providing instant alerts regarding meal approvals, allergy conflicts, and kitchen order updates.

Overall, these integrations and optimizations would significantly improve the functionality, performance, scalability, and usability of HIMPS, making it a more advanced and efficient solution for hospital nutrition and dietary management.

## CHAPTER -12 CONCLUSION

### 12.1 Summary of the Project

The **Hospital Inpatient Meal Planning System (HIMPS)** is a comprehensive web-based application developed to improve the management of patient nutrition and dietary services within hospitals. The project was designed to address the challenges associated with traditional paper-based meal planning systems, which often result in communication gaps, dietary errors, delayed meal delivery, and difficulties in monitoring patient nutritional requirements. By digitizing the entire meal planning process, HIMPS provides an efficient, reliable, and centralized solution for healthcare professionals involved in patient nutrition management.

The primary objective of the project was to create a platform that allows hospitals to manage patient dietary information, meal planning, allergy monitoring, calorie calculations, kitchen operations, and approval workflows from a single system. The application supports multiple user roles, including Administrators, Dietitians, Nurses, and Kitchen Staff, ensuring that each user can access only the functionalities relevant to their responsibilities. This role-based approach improves security, accountability, and operational efficiency.

One of the key achievements of the project is the implementation of a **Patient Management Module**, which enables healthcare staff to maintain accurate patient records, including medical conditions, ward details, and admission information. This module serves as the foundation for all nutrition-related activities within the system. The **Dietary Profile Module** further extends patient management by storing information about diet types, calorie requirements, allergies, food restrictions, and nutritional goals. These profiles help dietitians prepare personalized meal plans tailored to each patient's medical needs.

The project also includes a **Meal Library Module**, which maintains a centralized database of meals along with nutritional information, calorie values, diet compatibility, and allergen details. Using this information, dietitians can create balanced and medically appropriate meal schedules through the **Meal Planning Module**. The system automatically tracks calorie intake and ensures that meals align with patient dietary requirements.

Patient safety was a major consideration during development. To address this, an **Allergy Check Module** was implemented to compare patient allergy information with meal ingredients before meals are assigned. This feature helps prevent dietary conflicts and reduces the risk of allergic reactions. Additionally, the **Calorie Calculator Module** automates nutritional calculations based

on patient-specific factors such as age, gender, weight, height, and activity level, reducing manual effort and improving accuracy.

Another important feature of HIMPS is the **Kitchen Order Management Module**, which automatically generates food preparation requests from approved meal plans. Kitchen staff can track meal preparation and delivery status, ensuring efficient coordination between departments. The system also includes **Notifications, Approval Workflows, Reports, and Audit History**, which improve communication, transparency, and monitoring of daily operations.

From a technical perspective, the application was developed using **HTML5, CSS3, JavaScript, Node.js, and Express.js**. The frontend provides a responsive and user-friendly interface, while the backend handles business logic, authentication, API communication, and data management. JSON files and localStorage are used for data storage in the current implementation. RESTful APIs facilitate seamless communication between system components.

Extensive testing was conducted using tools such as Postman, Chrome Developer Tools, and Visual Studio Code debugging utilities to ensure system reliability and performance. Integration testing confirmed that all modules work together effectively, providing a stable and functional solution for hospital nutrition management.

In conclusion, the Hospital Inpatient Meal Planning System successfully demonstrates how modern web technologies can be applied to healthcare environments to improve patient nutrition services. The system reduces manual workload, improves patient safety, enhances communication among healthcare professionals, and supports efficient hospital operations. Through its integrated approach to dietary management, HIMPS contributes to better healthcare delivery and provides a strong foundation for future enhancements such as database integration, cloud deployment, mobile applications, artificial intelligence-based diet recommendations, and healthcare system interoperability.

## 12.2 What Was Achieved

The development of the **Hospital Inpatient Meal Planning System (HIMPS)** successfully achieved the primary goal of creating a centralized and digital solution for managing hospital nutrition and dietary services. The project transformed traditional meal planning procedures into an organized, efficient, and user-friendly web-based system capable of supporting healthcare professionals in their daily operations.

One of the major achievements of the project was the successful implementation of a **Patient Management System**. The application allows healthcare staff to maintain patient records, including personal information, medical conditions, ward details, bed assignments, and admission data. This created a structured foundation for all nutrition-related activities and improved the accessibility of patient information.

Another significant achievement was the development of the **Dietary Profile Management Module**. The system enables dietitians to create and manage patient-specific dietary profiles containing information such as diet type, calorie requirements, allergies, food restrictions, nutritional targets, and special dietary instructions. This feature supports personalized nutrition planning and ensures that patient dietary requirements are properly documented and monitored.

The project also successfully implemented a **Meal Library and Meal Planning System**. Dietitians can select meals from a centralized database containing nutritional information, calorie values, and allergen details. Daily meal plans can be generated efficiently while ensuring that nutritional goals and dietary restrictions are maintained. This significantly reduces the manual effort involved in meal planning.

Patient safety was enhanced through the successful implementation of an **Allergy Checking Mechanism**. The system automatically compares patient allergy information with meal allergen details and generates warnings whenever potential conflicts are detected. This achievement helps minimize dietary errors and improves the safety of meal assignments.

Another important accomplishment was the development of the **Calorie Calculator Module**, which automates nutritional calculations using patient-specific data. This feature improves accuracy and supports evidence-based dietary planning.

The system also achieved effective coordination between departments through the **Kitchen Order Management Module**, which automatically generates kitchen orders from approved meal plans and tracks preparation and delivery status. Additionally, features such as notifications, approval workflows, reporting, and audit history were successfully integrated to improve communication, transparency, and accountability.

From a technical perspective, the project successfully utilized **HTML5, CSS3, JavaScript, Node.js, and Express.js** to develop a complete full-stack web application. Overall, HIMPS achieved its objectives of improving patient nutrition management, reducing manual processes,

enhancing operational efficiency, and providing a reliable digital platform for hospital dietary services.

## 12.3 Skills Learned During Development

The development of the **Hospital Inpatient Meal Planning System (HIMPS)** provided valuable learning opportunities across various areas of software development, web technologies, problem-solving, testing, and project management. Throughout the project lifecycle, several technical and professional skills were acquired and enhanced, contributing significantly to overall knowledge and practical experience in full-stack web application development.

One of the major skills gained during the project was **frontend web development**. The use of **HTML5, CSS3, and JavaScript** helped in understanding how to create responsive and user-friendly web interfaces. Skills such as designing forms, tables, dashboards, navigation menus, and responsive layouts were improved. The project also provided experience in implementing dynamic user interactions, form validation, event handling, and interactive user interface components using JavaScript.

Another important area of learning was **backend development using Node.js and Express.js**. The project provided practical experience in creating server-side applications, handling client requests, processing data, and managing business logic. Knowledge of RESTful API development, routing mechanisms, middleware implementation, and server configuration was significantly enhanced through the development of various application modules.

The project also improved understanding of **database design and data management** concepts. Although JSON files and localStorage were used as the storage mechanism, valuable knowledge was gained regarding data organization, entity relationships, record management, and data retrieval techniques. Understanding how different modules interact and share information contributed to stronger system design skills.

Another significant skill acquired was **API integration and testing**. Working with REST APIs helped in understanding request-response communication between frontend and backend systems. Tools such as Postman were used to test API endpoints, validate responses, and identify errors. This experience improved debugging and troubleshooting abilities.

The project also enhanced **software testing and quality assurance skills**. Different testing approaches, including unit testing, integration testing, and functional testing, were performed to verify system behavior. The use of browser developer tools, debugging techniques, and error analysis strengthened problem-solving capabilities.

In addition, valuable experience was gained in **version control and project management** through the use of Git and GitHub. Skills such as source code management, change tracking, collaboration, and maintaining project history were developed during implementation.

Finally, the project improved soft skills such as teamwork, communication, documentation, time management, and analytical thinking. Coordinating multiple modules, solving technical challenges, and preparing project reports contributed to overall professional growth.

Overall, the development of HIMPS provided comprehensive exposure to modern web development technologies and software engineering practices, helping build both technical expertise and practical problem-solving skills essential for future projects and professional careers.

## CHAPTER -13 REFERENCES

### Books

Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.

Freeman, E., & Freeman, E. (2020). *Head First HTML and CSS* (2nd ed.). O'Reilly Media.

Duckett, J. (2014). *HTML & CSS: Design and Build Websites*. Wiley.

Duckett, J. (2015). *JavaScript and JQuery: Interactive Front-End Web Development*. Wiley.

Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.

### Tutorials and Learning Resources

W3Schools. (2025). *HTML, CSS, and JavaScript Tutorials*. Retrieved from <https://www.w3schools.com>

MDN Web Docs. (2025). *Web Technologies Documentation*. Retrieved from <https://developer.mozilla.org>

FreeCodeCamp. (2025). *Responsive Web Design Certification*. Retrieved from <https://www.freecodecamp.org>

GeeksforGeeks. (2025). *Front-end Development and Web Technologies*. Retrieved from <https://www.geeksforgeeks.org>

Tutorialspoint. (2025). *HTML, CSS, JavaScript, and Web Development Tutorials*. Retrieved from <https://www.tutorialspoint.com>

### APIs and Tools Used

Google Maps Platform. (2025). *Maps JavaScript API Documentation*. Retrieved from <https://developers.google.com/maps>

Unsplash API. (2025). *Free High-Resolution Image Access*. Retrieved from <https://unsplash.com/developers>

OpenWeatherMap. (2025). *Weather API for Travel Assistance*. Retrieved from <https://openweathermap.org/api>

RapidAPI Hub. (2025). *Travel and Tourism APIs*. Retrieved from <https://rapidapi.com/hub>

Firebase by Google. (2025). *Authentication and Database Services*. Retrieved from <https://firebase.google.com>

### Documentation and Research References

World Tourism Organization (UNWTO). (2024). *Tourism Data Dashboard*. Retrieved from <https://www.unwto.org>

Booking.com. (2025). *Accommodation and Destination Insights*. Retrieved from <https://www.booking.com>

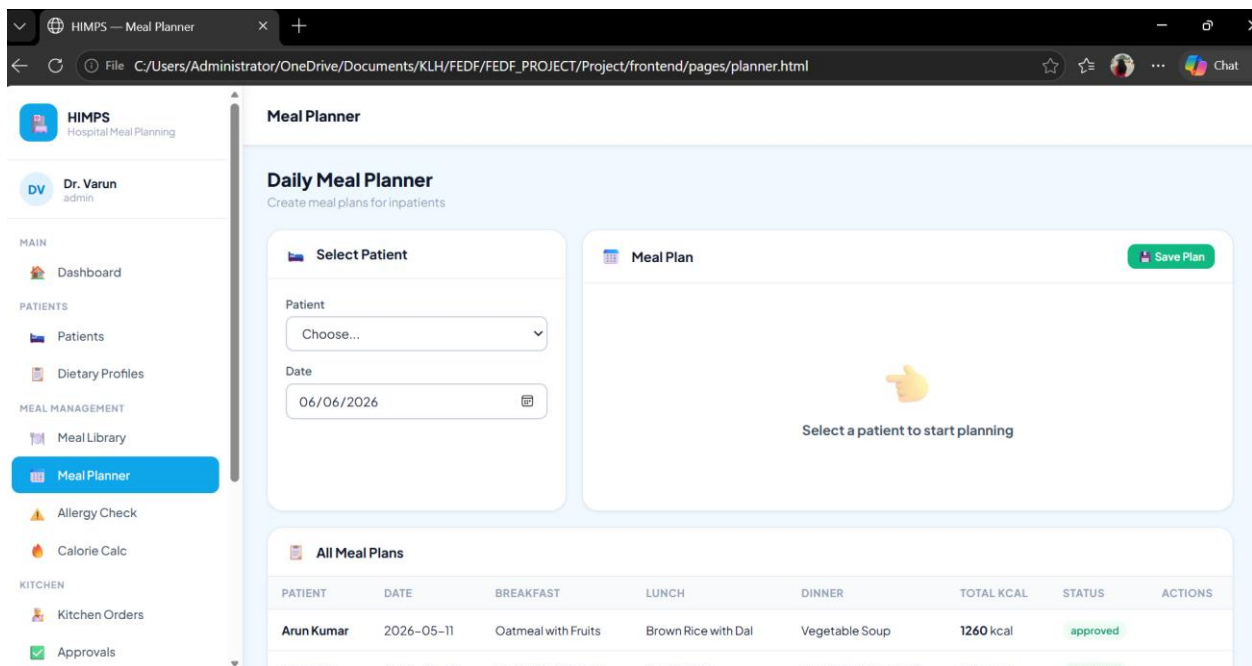
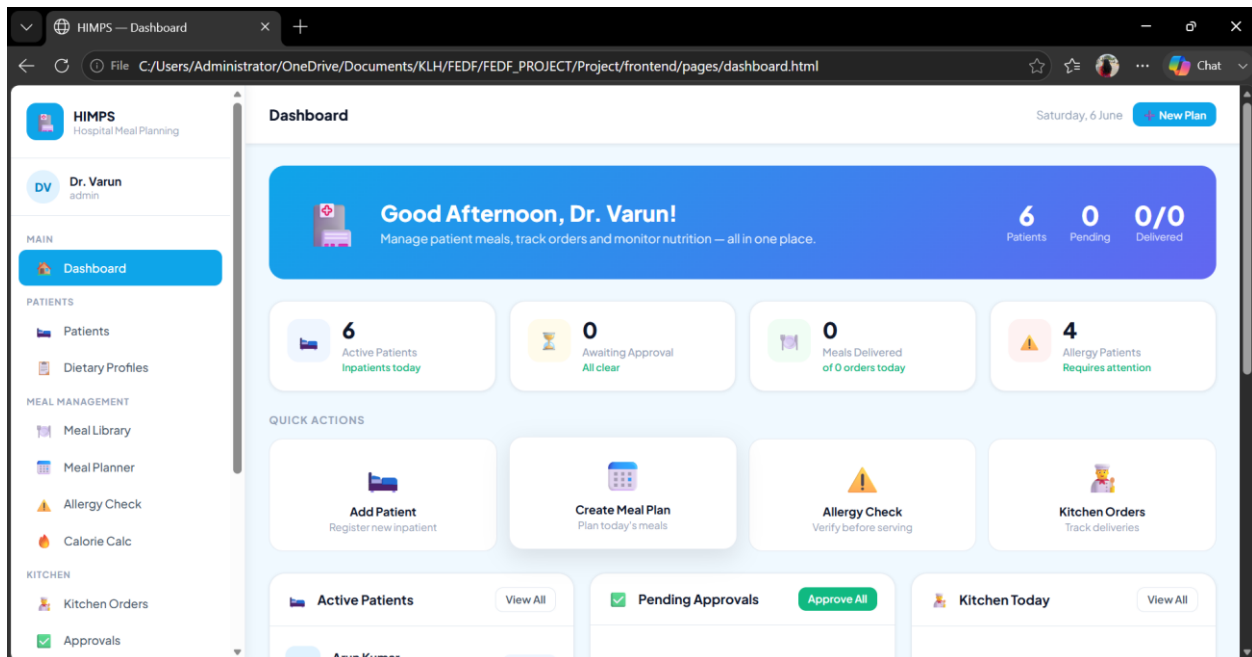
TripAdvisor. (2025). *Tourism and Destination Review Data*. Retrieved from <https://www.tripadvisor.com>

Travel + Leisure. (2025). *Top Global Destinations and Travel Trends*. Retrieved from <https://www.travelandleisure.com>

Tourism Review. (2025). *Digital Transformation in the Tourism Industry*. Retrieved from <https://www.tourism-review.com>

## CHAPTER -14 APPENDICES

### Screenshots of the app :



HIMPS — Allergy Check

C:/Users/Administrator/OneDrive/Documents/KLH/FEDF/FEDF\_PROJECT/Project/frontend/pages/allergy.html

HIMPS

Hospital Meal Planning

Dr. Varun

admin

MAIN

Dashboard

PATIENTS

Patients

Dietary Profiles

MEAL MANAGEMENT

Meal Library

Meal Planner

Allergy Check

Calorie Calc

KITCHEN

Kitchen Orders

Approvals

Allergy Check

Verify meals against patient allergies before serving

Check Patient vs Meal

Select Patient

Choose patient...

Select Meal

Choose meal...

Check Allergy

High Risk Patients

Arun Kumar

Gluten

Meena Devi

Shellfish

Lakshmi Bai

Nuts

Harshika

Fish

All Patient Allergy Profiles

| PATIENT    | WARD/BED    | DIET TYPE | ALLERGIES | RESTRICTIONS         |
|------------|-------------|-----------|-----------|----------------------|
| Arun Kumar | Ward A/A-12 | Diabetic  | Gluten    | High Sugar, High Fat |

HIMPS — Calorie Calculator

C:/Users/Administrator/OneDrive/Documents/KLH/FEDF/FEDF\_PROJECT/Project/frontend/pages/calorie.html

HIMPS

Hospital Meal Planning

Dr. Varun

admin

MAIN

Dashboard

PATIENTS

Patients

Dietary Profiles

MEAL MANAGEMENT

Meal Library

Meal Planner

Allergy Check

Calorie Calc

KITCHEN

Kitchen Orders

Approvals

Calorie Calc

Calculate and track daily caloric requirements

BMR & Calorie Calculator

Weight (kg)

e.g. 70

Height (cm)

e.g. 170

Age

e.g. 45

Gender

Male

Activity Level

Sedentary (bed rest)

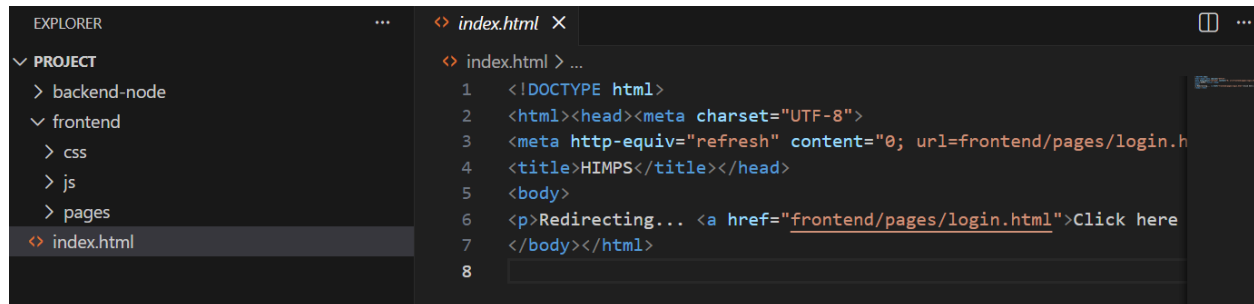
Medical Condition

Normal

Patient Calorie Summary

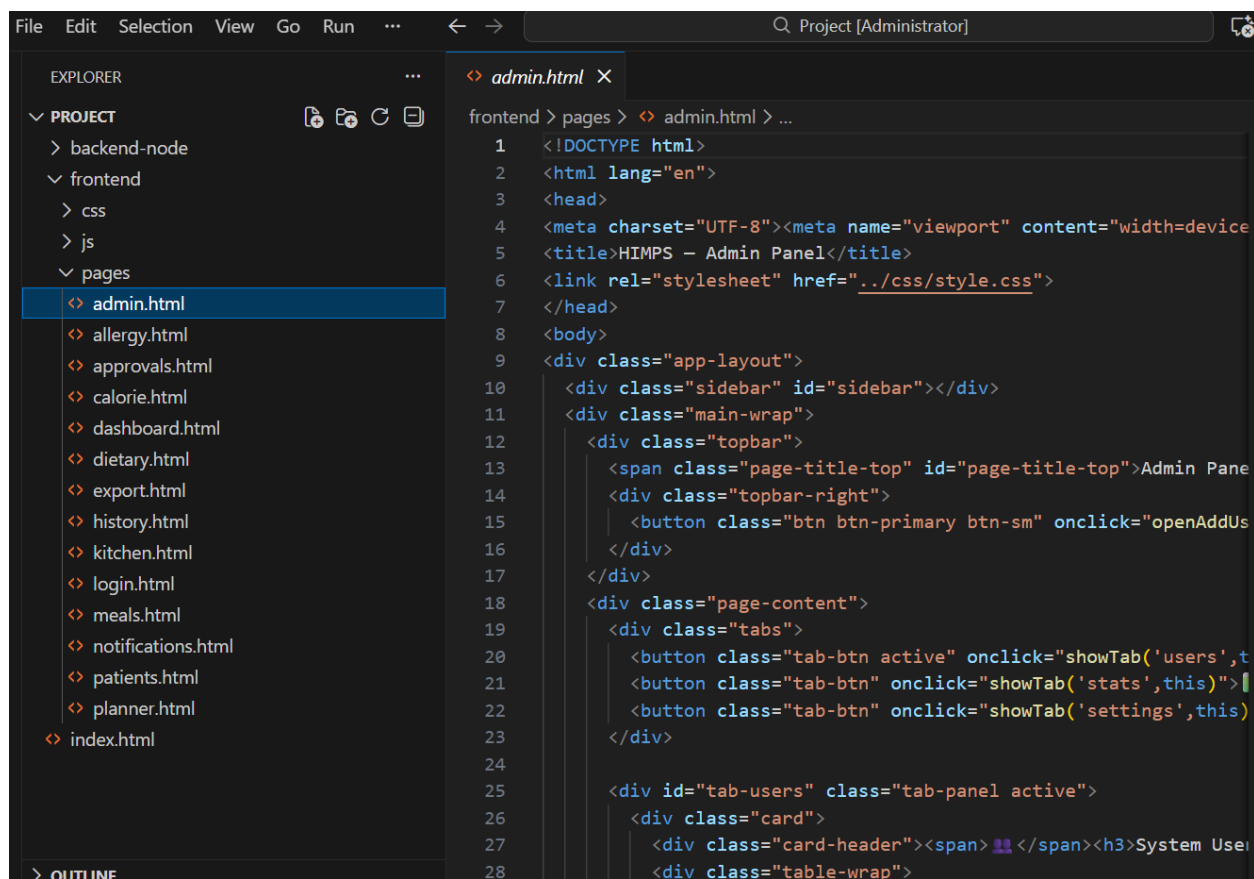
|                  |             |
|------------------|-------------|
| Arun Kumar       | 0/1800 kcal |
| Meena Devi       | 0/1600 kcal |
| Suresh Reddy     | 0/2200 kcal |
| Lakshmi Bai      | 0/1700 kcal |
| Harshika         | 0/2000 kcal |
| Puthana Praveena | 0/0 kcal    |

## Sample code snippets :



This screenshot shows the Visual Studio Code editor with the Explorer sidebar on the left. The 'PROJECT' folder is expanded, showing subfolders 'backend-node', 'frontend', 'css', 'js', and 'pages'. The 'index.html' file is selected and opened in the main editor. The code in 'index.html' is as follows:

```
1 <!DOCTYPE html>
2 <html><head><meta charset="UTF-8">
3 <meta http-equiv="refresh" content="0; url=frontend/pages/login.h
4 <title>HIMPS</title></head>
5 <body>
6 <p>Redirecting... <a href="frontend/pages/login.html">Click here
7 </body></html>
8
```



This screenshot shows the Visual Studio Code editor with the Explorer sidebar on the left. The 'PROJECT' folder is expanded, and the 'pages' subfolder is selected. The 'admin.html' file is selected and opened in the main editor. The code in 'admin.html' is as follows:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8"><meta name="viewport" content="width=device
5 <title>HIMPS - Admin Panel</title>
6 <link rel="stylesheet" href=" ../css/style.css">
7 </head>
8 <body>
9 <div class="app-layout">
10 <div class="sidebar" id="sidebar"></div>
11 <div class="main-wrap">
12 <div class="topbar">
13 <span class="page-title-top" id="page-title-top">Admin Pane
14 <div class="topbar-right">
15 <button class="btn btn-primary btn-sm" onclick="openAddUs
16 </div>
17 </div>
18 <div class="page-content">
19 <div class="tabs">
20 <button class="tab-btn active" onclick="showTab('users',t
21 <button class="tab-btn" onclick="showTab('stats',this)">
22 <button class="tab-btn" onclick="showTab('settings',this)
23 </div>
24
25 <div id="tab-users" class="tab-panel active">
26 <div class="card">
27 <div class="card-header"><span>👤</span><h3>System User
28 <div class="table-wrap">
```

File Edit Selection View Go Run ... < > Q Project [Administrator]

EXPLORER

PROJECT

backend-node

frontend

css

js

app.js

sidebar.js

pages

admin.html

allergy.html

approvals.html

calorie.html

dashboard.html

dietary.html

export.html

history.html

kitchen.html

login.html

meals.html

notifications.html

patients.html

planner.html

index.html

OUTLINE

TIMELINE

JS sidebar.js X

frontend > js > JS sidebar.js > renderSidebar

1 function renderSidebar(activePage) {  
2 Auth.guard();  
3 const user = Auth.session();  
4 const unread = Utils.unreadCount();  
5  
6 const allLinks = [  
7 { label:'MAIN', items:[  
8 { page:'dashboard', icon:'🏠', label:'Dashboard',  
9 ]},  
10 { label:'PATIENTS', items:[  
11 { page:'patients', icon:'👤', label:'Patients',  
12 { page:'dietary', icon:'📋', label:'Dietary Profiles',  
13 ]},  
14 { label:'MEAL MANAGEMENT', items:[  
15 { page:'meals', icon:'🍽️', label:'Meal Library',  
16 { page:'planner', icon:'📅', label:'Meal Planner',  
17 { page:'allergy', icon:'⚠️', label:'Allergy Check',  
18 { page:'calorie', icon:'🔥', label:'Calorie Calc',  
19 ]},  
20 { label:'KITCHEN', items:[  
21 { page:'kitchen', icon:'👨🍳', label:'Kitchen Orders',  
22 { page:'approvals', icon:'✅', label:'Approvals',  
23 ]},  
24 { label:'REPORTS', items:[  
25 { page:'history', icon:'📄', label:'History',  
26 { page:'export', icon:'📤', label:'Export',  
27 { page:'notifications', icon:'🔔', label:'Notifications',  
28 ]},  
29 { label:'ADMIN', items:[

## Installation/setup instructions :

The **Hospital Inpatient Meal Planning System (HIMPS)** is a web-based application developed using HTML5, CSS3, JavaScript, Node.js, and Express.js. To run the application successfully, the development environment must be configured properly. The following steps describe the installation and setup process.

### Step 1: Install Required Software

Before running the project, install the following software on your system:

- Node.js (Version 18 or above)
- npm (Node Package Manager)
- Visual Studio Code (Optional)
- Google Chrome or any modern web browser
- Git (Optional)

### Step 2: Download the Project

Download or clone the HIMPS project files from the project repository and store them in a local directory. Ensure that all frontend, backend, and configuration files are present.

### Step 3: Open the Project

Open the project folder using Visual Studio Code or any preferred code editor. Navigate to the project root directory where the package configuration file is located.

### Step 4: Install Dependencies

Open the terminal and execute the following command:

```
npm install
```

This command installs all required project dependencies, including Express.js, CORS, and other supporting libraries.

### **Step 5: Configure the Application**

Verify that all configuration files, JSON data files, and project folders are correctly placed. Ensure that the server configuration and file paths are properly defined.

### **Step 6: Start the Application**

Run the backend server using:

```
npm start
```

or

```
node server.js
```

The server will start and listen for incoming requests.

### **Step 7: Access the System**

Open a web browser and navigate to:

```
http://localhost:3000
```

The HIMPS login page will appear. Enter valid user credentials to access the system.

### **Step 8: Verify Functionality**

After login, test major modules such as Patient Management, Dietary Profiles, Meal Planning, Kitchen Orders, Notifications, and Reports to ensure proper operation.

Following these steps will successfully install and configure the Hospital Inpatient Meal Planning System for development, testing, and deployment purposes.

## **Geo Tag photos with guide**

**Review forms with guide signatures**

